FINAL PROJECT REPORT ▪ RESEARCH IN INFORMATION SECURITY

Cutting the Cord

A Three-Tier, Authority-Independent Authentication and Key Management Framework for TUAV-Assisted Infrastructure-Less IoV

Analysis, design, formal verification, implementation and measured evaluation

Swaraj Kumar (2025202016) **Hritik Ranjan** (2025202031) **Harsh Raj** (2025202005)

Team 6 ▪ October 2026

Base paper: H. Tan, W. Zheng and P. Vijayakumar, "Secure and efficient authenticated key management scheme for UAV-assisted infrastructure-less IoVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 6389–6400, 2023.

What this report delivers

Analysis	six findings against the base scheme; four reproduced in code, one confirmed by ProVerif
Design	three tiers (UAVs, vehicles, multi-TUAV handover) under one *offline* trust root
Formal verification	five ProVerif models, 16 queries, every result as predicted
Implementation	both schemes on the same curve (BN254), 20 passing tests
Evaluation	eight experiments (E0–E7), measured and analytical

Abstract

Tan, Zheng and Vijayakumar proposed a tethered UAV (TUAV) that takes the place of destroyed roadside units so that an Internet of Vehicles (IoV) keeps working after a disaster. We studied the scheme equation by equation, implemented it, and found that it does not achieve the property it is named for. Every authentication is relayed over a satellite link to a cloud Trusted Authority (TA) that performs all pairings, so authentication stops when the link is lost. The UAV credential depends only on values the TA holds, so the TA can impersonate any UAV; ProVerif finds this attack automatically. The CRT-based group key leaks every member's decrypting key to every other member, so a revoked UAV still obtains the new key; our implementation reproduces the attack. The reported cost and traffic omit the TA's pairings and a group-key broadcast that grows as $O(n^3)$ bytes (17.5 MB for 120 UAVs under our encoding).

We propose **Cutting the Cord**, which keeps the TUAV architecture but makes the trust root *offline*. **C1**: the TUAV authenticates UAVs itself with escrow-free, pairing-free certificateless signatures in a small-exponent batch with a bounded fallback, and delivers the group key per member under fresh ECDH keys. **C2**: vehicles, which the base paper leaves out, are admitted through authenticated UAVs that relay and filter their requests. **C3**: neighbouring TUAVs hand members over with single-use encrypted tickets.

ProVerif verifies authentication, key secrecy and forward secrecy for C1 and C3 and revocation for rekeying. It also delimits the escrow guarantee precisely: a malicious TA cannot sign under a registered key but, as in every certificateless scheme, can enrol a replacement key. On the same curve and library, our TUAV authenticates and keys n UAVs one to two orders of magnitude faster than the base scheme's TA-inclusive cost. It needs no satellite traffic, keeps admitting honest UAVs when a batch is poisoned, and reduces key-delivery traffic from megabytes to kilobytes. With the base paper's own MIRACL constants the analytical figures are 228 ms against 3720 ms (plus the satellite round trip) for 120 UAVs.

Keywords: infrastructure-less IoV, tethered UAV, certificateless authentication, batch verification, group key management, revocation, handover, ProVerif, conditional privacy.

How to Read This Report

Five kinds of coloured box appear throughout:

What this step does	the purpose of a protocol step, in plain words
Why it is designed this way	the reason behind each design choice
Finding	a flaw we found in the base paper
Key idea	the one sentence to remember
Honest limitation	what our proposal does *not* solve

Colours are consistent: **dark blue** = Trusted Authority, **teal** = UAV tier (C1), **violet** = vehicle tier (C2), **amber** = TUAV handover (C3), **red** = flaws and attacks, **green** = properties that hold. Chapters 3–4 use the *base paper's* symbols ($\sigma_i, \Gamma_i, \kappa_{tu}, \Lambda(x)$); the rest of the report uses *our* symbols (Table 6.1). In particular, σ_i always means the base paper's decrypting key, and our signatures are written (R, v).

Three kinds of evidence. Every claim is marked with how we know it: *analytical* (operation counts times published timings, the method of the base paper), *measured* (our implementation, Chapter 16), or *formally verified* (ProVerif, Chapter 13). Proofs in Chapter 12 are proof sketches that follow standard reductions.

Contents

Abstract 1

How to Read This Report 2

1 Introduction 6
- 1.1 Motivation . . . 6
- 1.2 The base paper . . . 6
- 1.3 Problem statement . . . 6
- 1.4 Summary of findings . . . 6
- 1.5 Contributions . . . 7
- 1.6 Headline results . . . 7
- 1.7 Organisation . . . 7

2 Background and Related Work 8
- 2.1 Authentication in the Internet of Vehicles . . . 8
- 2.2 Certificateless public-key cryptography . . . 8
- 2.3 Batch verification and its failure modes . . . 8
- 2.4 Group key management . . . 8
- 2.5 UAV-assisted vehicular networks . . . 9
- 2.6 Formal verification with ProVerif . . . 9
- 2.7 Positioning . . . 9

3 The Base Scheme in Brief 10

4 Critical Analysis of the Base Scheme 11
- 4.1 Finding 1: the scheme is not infrastructure-less . . . 11
- 4.2 Finding 2: the TA can impersonate every UAV . . . 11
- 4.3 Finding 3: no forward secrecy . . . 12
- 4.4 Finding 4: the CRT group key leaks, and revocation fails . . . 12
- 4.5 Finding 5: the unforgeability proof is empty . . . 13
- 4.6 Finding 6: the cost accounting leaves out the expensive part . . . 14
- 4.7 Summary of findings . . . 14

5 System Model, Threat Model and Design Goals 15
- 5.1 System model . . . 15
- 5.2 Threat model . . . 15
- 5.3 Design goals . . . 16

6 Preliminaries and Notation 17

7 Proposed Framework: Setup and Offline Registration 19
- 7.1 System setup (ST) . . . 19
- 7.2 Entity registration (RG) . . . 19

8 C1: Autonomous UAV Authentication and Group Key Management 21
- 8.1 Steps . . . 21
- 8.2 When a batch fails: bounded fallback . . . 23
- 8.3 Rekeying on leave, eviction and join (RK) . . . 24

9 C2: The Vehicle Tier **25**
9.1 Steps . . . 25
9.2 V2V communication under the regional key . . . 26

10 C3: TUAV-to-TUAV Handover **27**
10.1 Steps . . . 27

11 The TA's Offline Role: Tracing and Global Revocation **29**

12 Security Analysis **30**
12.1 Authority independence . . . 30
12.2 Unforgeability, escrow-freeness and non-repudiation . . . 30
12.3 Batch soundness . . . 31
12.4 Mutual authentication and key secrecy . . . 31
12.5 Privacy . . . 31
12.6 Replay and handover security . . . 32
12.7 Security comparison . . . 32

13 Formal Verification with ProVerif **33**
13.1 Modelling choices . . . 33
13.2 Results . . . 34
13.3 Interpretation . . . 34

14 Implementation **36**
14.1 Design principles . . . 36
14.2 Structure . . . 36
14.3 Realising the base scheme . . . 36
14.4 Realising our framework . . . 37
14.5 Tests . . . 37

15 Analytical Comparison with the Base Paper's Constants **39**
15.1 Computation . . . 39
15.2 Communication . . . 41
15.3 Handover . . . 41

16 Measured Evaluation **43**
16.1 Setup and methodology . . . 43
16.2 E0: primitive costs . . . 43
16.3 E1: availability under TA-link loss . . . 44
16.4 E2: time to authenticate and key a swarm . . . 44
16.5 E3: batch poisoning . . . 45
16.6 E4: communication . . . 46
16.7 E5: the vehicle tier . . . 46
16.8 E6: handover . . . 47
16.9 E7: attacks, executed . . . 47
16.10Summary . . . 47

17 Discussion, Limitations and Conclusion **48**
17.1 What the evidence supports . . . 48
17.2 Limitations . . . 48
17.3 Conclusion . . . 49
17.4 Future work . . . 49

Bibliography **50**

A Reproducing the Results **52**

B ProVerif Models **53**
B.1 Shared library . . . 53
B.2 C1 against an outsider (Type I) . . . 53
B.3 C1 against a malicious TA (Type II) . . . 54
B.4 Rekeying . . . 55
B.5 C3 handover . . . 55
B.6 The base scheme's credential . . . 56
B.7 Raw results . . . 57

List of Figures

3.1 Base scheme [1]. Red arrows cross the satellite link. GA3 cannot run until GA2 returns. 10

4.1 Revocation break (Finding 4): an insider learns every decrypting key from the public coefficients and later uses a staying member's key to open the updated polynomial. . 13

5.1 The three-tier framework. The TA (top) acts only before deployment and for later tracing; all field operations (C1–C3) run without it. 15

7.1 Phases of the framework and their step labels. 19

8.1 C1 message flow. Compare with Figure 3.1: the satellite round trip and the TA's pairings are gone. 21
8.2 Adaptive verification rule of Step UA3. 23

9.1 C2: vehicle authentication relayed by an authenticated UAV. 25

10.1 C3 handover. Only symmetric operations and one ECDH are needed at the new TUAV. 27

15.1 Group authentication cost versus n (satellite RTT *not* added to the base scheme here). Counted honestly, the base scheme is the most expensive of all six. 40
15.2 Left: end-to-end time with the satellite round trip charged to the base scheme. Right: *analytical* availability. The base scheme falls to zero with the link; ours does not depend on it (Proposition 2). 40
15.3 Group-key delivery traffic. The base scheme grows as $O(n^3)$ bytes; ours grows linearly, and at $n = 120$ is about 2150× smaller. 41

16.1 Share of authentication rounds that succeed as TA-link availability falls (20 000 rounds per point), for independent losses and for bursty outages (Gilbert–Elliott, mean outage 10 rounds). 44
16.2 Left: measured time for the TUAV to authenticate and key n UAVs (ours) against the base scheme's GA1–GD4 with the TA's work included, and the TUAV-only part the base paper reports. Right: load-independent estimate from operation counts. Log scale; satellite RTT not added. The measured $n = 120$ point ran across a change in machine speed (see text). 44
16.3 Left: share of honest UAVs admitted as the share of invalid requests grows ($n = 60$). Right: verification time of three strategies for handling a failed batch. 45
16.4 Group-key traffic and the base scheme's satellite traffic, measured from the encodings of our implementation (compressed points, 32-byte hashes). 46
16.5 TUAV time to admit a bundle of vehicles relayed by one UAV (batch verification *and* key delivery) against verifying the same requests one by one. 46
16.6 Computation for one handover: full re-admission in the new cell against a C3 ticket (median of 25 runs). 47

List of Tables

4.1 Measured size of the CRT coefficient set (`scripts/compute_results.py`). 13
4.2 Our findings mapped to the base paper's claims. 14

6.1 Notation of the proposed framework (our symbols). 18

8.1 Verification time at the TUAV for $n = 120$ with d invalid requests (average of 200 random placements; `compute_results.py`). 23

12.1 Security properties. ✓ holds, ✗ fails, ● partially. "Base (claimed)" is Table II of [1]; "Base (actual)" is our analysis. 32

13.1 ProVerif results. "Expected" is what the design predicts; every result matches it. . . 34

14.1 Code layout. 36
14.2 The 20 tests (`python -m pytest`, all passing). 37

15.1 Computation cost (ms). "Base (honest)" adds the TA's GA2 work (29.7414 ms per UAV) and places Γ_i in $\mathbb{G}_1$. 39
15.2 Communication overhead (bytes). 41
15.3 Admission in a new cell: full C1/C2 admission versus a C3 ticket. 41

16.1 Median cost of primitives on the measurement machine (E0). 43
16.2 E2 measurements (ms). Our time includes verification *and* key delivery. The base scheme's time is split into TA (GA2), TUAV (GA3) and key distribution (GD). . . . 45
16.3 Attack outcomes, executed against both implementations. 47

CHAPTER 1

Introduction

1.1 Motivation

In a normal Internet of Vehicles (IoV), roadside units (RSUs) on poles verify every vehicle and relay data to a cloud Trusted Authority (TA). After an earthquake or flood the poles, the fibre and the power are the first things to fail. Rescue convoys and evacuation traffic then need secure coordination most, and nobody is left to verify anyone. Tan, Zheng and Vijayakumar [1] call this the *infrastructure-less IoV*. Their answer is a *tethered UAV* (TUAV): a drone powered by cable from an emergency communications response vehicle (ECRV), which hovers for hours and acts as a flying edge server for a swarm of ordinary UAVs.

1.2 The base paper

The base paper makes three contributions:

(i) a TUAV-enabled edge architecture that replaces the dead RSU and gives three-dimensional coverage;

(ii) certificateless *group* authentication of n UAVs at once, with the costly bilinear pairings offloaded to the TA;

(iii) a group key distributed with the Chinese Remainder Theorem (CRT) and updated cheaply when UAVs join or leave.

It reports 151 ms to authenticate 120 UAVs, 104 bytes per request, and all eleven security properties of its comparison table.

1.3 Problem statement

Research question

Does the published scheme actually work *without infrastructure*, and does it deliver the security properties it claims? If not, can those properties be recovered together, at comparable cost, and can the same design be extended to the vehicles and to a multi-TUAV deployment?

1.4 Summary of findings

We re-derived every equation of the base paper, implemented it on a real pairing-friendly curve, and modelled its credential in ProVerif. The architecture is a good idea, but the protocol has six problems (Chapter 4):

F1. It is not infrastructure-less. The TUAV cannot verify anyone itself; every authentication needs a satellite round trip to the TA.

F2. The TA can impersonate every UAV (no escrow resilience, no non-repudiation). Our code reproduces this, and ProVerif finds it automatically.

F3. No forward secrecy: one leaked long-term secret exposes every past group key.

F4. Revocation does not revoke: the CRT polynomial reveals every member's key, so a removed UAV still decrypts. Our code reproduces this.

F5. The unforgeability proof is empty, and the batch check accepts cancelling errors. Our code reproduces the cancellation.

F6. The cost accounting omits the TA's pairings and an $O(n^3)$-byte key broadcast. We measured both.

1.5 Contributions

Key idea

Keep the TUAV architecture and move the trust root *offline*. The TA certifies credentials before deployment and can trace a misbehaving entity afterwards, but no operation in the field ever waits for it. Every UAV, vehicle and TUAV can verify every other one using only the TA's public key.

1. **Analysis** of the base scheme: six findings, four reproduced in code and one confirmed by ProVerif (Chapter 4).
2. **C1, autonomous UAV tier**: escrow-free, pairing-free certificateless credentials verified locally by the TUAV in a small-exponent batch with a bounded fallback; per-member group-key delivery under fresh ECDH keys; secure rekeying (Chapter 8).
3. **C2, vehicle tier**: vehicles admitted through authenticated UAVs that relay and filter their requests, with a separate regional key for V2V (Chapter 9).
4. **C3, multi-TUAV handover**: single-use encrypted tickets and one fresh ECDH, with no signature verification and no TA (Chapter 10).
5. **Formal verification** in ProVerif: five models and sixteen queries, including the base scheme's escrow attack and the exact boundary of our own escrow guarantee (Chapter 13).
6. **Implementation and measured evaluation**: both schemes on the same curve and library, 20 tests, and eight experiments covering availability, latency, batch poisoning, traffic, vehicles, handover under mobility, and attacks (Chapters 14–16).

1.6 Headline results

Metric	Base scheme	Ours	Evidence
Authentication when the TA link is down	0%	100%	measured model (E1)
TA can impersonate a registered UAV	yes	no	ProVerif + code (E7)
Revoked UAV obtains the new group key	yes	no	ProVerif + code (E7)
Honest UAVs admitted with one bad request	0%	all	code (E3, E7)
Group-key traffic, $n = 120$	17.5 MB	9.6 KB	measured (E4)
Authenticate + key 120 UAVs, base paper's constants	3720 ms + RTT	228 ms	analytical

1.7 Organisation

Chapter 2 reviews the background. Chapter 3 recalls the base scheme and Chapter 4 analyses it. Chapter 5 gives the system and threat model, the design goals and the notation. Chapters 7–11 present the framework, Chapter 12 the security analysis and Chapter 13 the formal verification. Chapter 14 describes the implementation. Chapters 15 and 16 report the analytical and the measured evaluation. Chapter 17 discusses limitations and concludes.

CHAPTER 2

Background and Related Work

2.1 Authentication in the Internet of Vehicles

IoV authentication schemes protect vehicle-to-infrastructure (V2I) and vehicle-to-vehicle (V2V) messages sent over open radio channels. Most of them share three design patterns. (i) A *trusted authority* registers vehicles and can trace a pseudonym back to a real identity (*conditional privacy*). (ii) *Roadside units* verify messages locally. (iii) *Batch verification* checks many signatures at once. The four schemes the base paper compares against are representative: Mei *et al.* [2], Kumar *et al.* [3], Ali *et al.* [4] and Xu *et al.* [5] are certificateless (aggregate) signature schemes for VANETs. All of them assume a working RSU or an online authority. The base paper [1] is the first to assume neither RSUs *nor* terrestrial backhaul. As we show, however, it still assumes an online authority reached by satellite.

2.2 Certificateless public-key cryptography

Identity-based cryptography removes certificates but gives the key generation centre (KGC) every private key (*key escrow*). Certificateless cryptography [6] splits the private key: the KGC issues a *partial* key bound to the identity, and the user adds a *secret value* the KGC never sees. Two adversaries are standard. A *Type-I* outsider may replace public keys but does not know the master key. A *Type-II* malicious KGC knows the master key but may not replace public keys. Girault's trust levels [7] make the residual trust precise. At *level 1* the authority knows users' secret keys and can impersonate them undetectably. At *level 2* it does not know them, but it can still impersonate a user by issuing a *second* public key, and the two coexisting keys expose the cheating. At *level 3* even that is impossible. Certificateless schemes reach level 2. This distinction matters in Chapter 13: the base scheme is at level 1 (Finding 2); ours is at level 2, and ProVerif confirms exactly that boundary. Pairing-free certificateless signatures such as ECLAS [8] avoid bilinear pairings entirely and verify with a few elliptic-curve scalar multiplications.

2.3 Batch verification and its failure modes

Bellare, Garay and Rabin [11] showed that checking a product (or sum) of n verification equations is unsound unless each equation is first multiplied by a random *small exponent*. Without them, errors in two signatures can cancel each other. When a batch fails, the bad signatures must be found. Binary splitting and group-testing strategies [12, 13, 14] do this with $O(d \log n)$ re-tests for d bad signatures, under the assumption that the verifier itself performs the re-tests cheaply. Two consequences for this project: (a) the base scheme's batch equation has no random exponents (Finding 5); (b) in the base scheme the verifier is *not* the party that computes, because every re-test needs the TA and a satellite round trip.

2.4 Group key management

A group controller must send a fresh group key to all members and exclude leavers. Three families exist:

- *Unicast*: the key is encrypted separately for each member, so traffic is linear and the scheme is simple and easy to reason about.

- *Logical key hierarchy* (LKH) [15, 16]: a tree of key-encrypting keys, so one leave costs $O(\log n)$ ciphertexts.
- *Single-broadcast CRT schemes*, descending from the "secure lock" of Chiou and Chen [17]: one CRT value hides the key for every member. These need pairwise-coprime moduli and careful handling of leakage, because the public value is a function of every member's secret.

The base scheme uses a polynomial CRT variant. Finding 4 shows that its specific construction leaks every member's modulus to every other member. We replace it with unicast delivery (optionally LKH).

2.5 UAV-assisted vehicular networks

UAVs extend vehicular coverage in three dimensions. Related work includes certificateless signcryption for UAV association [18] and lightweight blockchain data sharing for disaster rescue [19]. These works, like the base paper, either assume ground infrastructure or a single aerial controller. To our knowledge, none provides authentication for both UAVs and vehicles, plus inter-controller handover, under an *offline* trust root.

2.6 Formal verification with ProVerif

ProVerif [23, 24] analyses protocols in the applied pi calculus against an unbounded Dolev–Yao attacker [22]. It proves secrecy ("`not attacker(M)`"), authentication as correspondence between events ("`event(A) ==> event(B)`", injective if each A needs a distinct B), and forward secrecy through *phases* in which long-term keys are leaked. When a property fails, ProVerif reconstructs an attack trace. It models cryptography symbolically (perfect primitives), so its results complement rather than replace computational reductions.

2.7 Positioning

Work	No RSU	No online TA	Escrow level	Vehicles	Multi-controller
VANET CL schemes [2, 3, 4, 5]	✗	✗	2 (claimed)	✓	RSU handover
Base paper [1]	✓	✗	1	✗	✗
This work	✓	✓	2	✓	✓

CHAPTER 3

The Base Scheme in Brief

This section uses the base paper's notation. Figure 3.1 shows the message flow we presented in Phase 1.

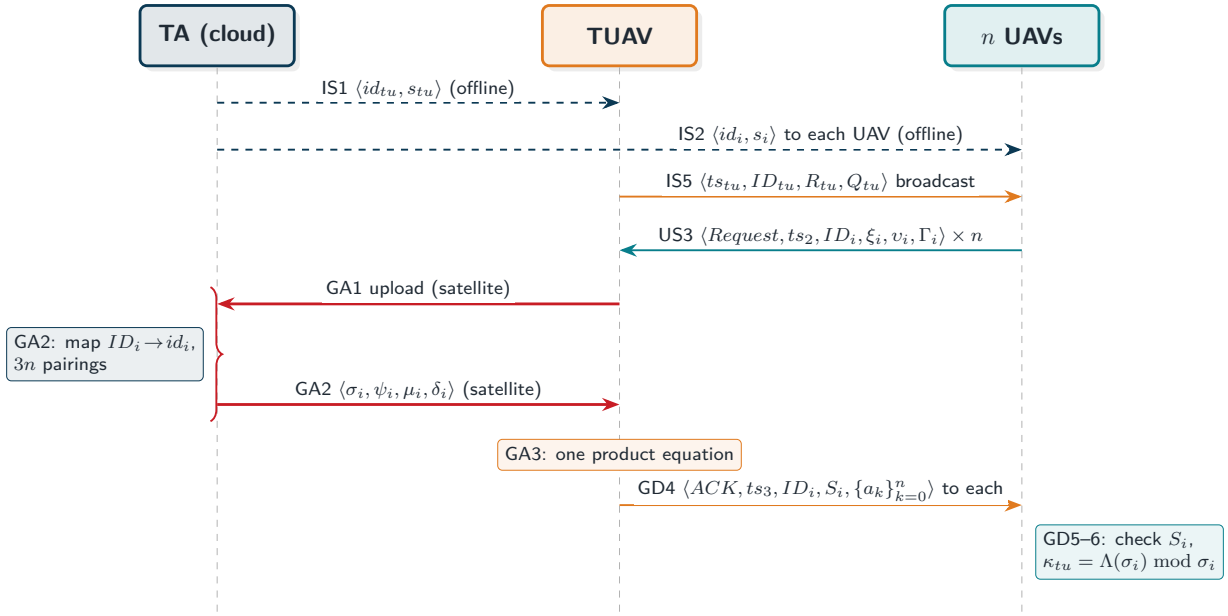


Figure 3.1: Base scheme [1]. Red arrows cross the satellite link. GA3 cannot run until GA2 returns.

The equations we use later (verbatim from [1], Section IV):

$$\text{IS4:} \quad ID_{tu} = h_2(id_{tu}, r_{tu}), \quad R_{tu} = r_{tu}\, h_2(ts_{tu}, s_{tu}) \in \mathbb{Z}_q, \quad Q_{tu} = s_{tu}\, h_2(ID_{tu}, r_{tu})\, P \tag{3.1}$$

$$\text{US1:} \quad \xi_i = h_1(r_i), \qquad ID_i = h_3(id_i, ts_2, \xi_i) \tag{3.2}$$

$$\text{US2:} \quad \upsilon_i = h_4(ID_i, ID_{tu}, ts_2, \xi_i), \qquad \Gamma_i = \big[s_i\, h_3(ID_i, ts_2, s_i)\, R_{tu} - h_1(s_i\xi_i)\big]P - \upsilon_i\, h_1(r_i)\, Q_{tu} \tag{3.3}$$

$$\text{GA2:} \quad \sigma_i = h_1(s_i\xi_i)P, \quad \psi_i = \hat{e}(\Gamma_i + \sigma_i, P), \quad \mu_i = \hat{e}(h_3(ID_i, ts_2, s_i)P, s_iP), \quad \delta_i = \hat{e}(s_{tu}P, \upsilon_i\xi_iP) \tag{3.4}$$

$$\text{GA3:} \quad \prod_i \psi_i \left(\prod_i \delta_i\right)^{h_2(ID_{tu}, r_{tu})} \stackrel{?}{=} \prod_i \mu_i^{R_{tu}} \tag{3.5}$$

$$\text{GD1–2:} \quad \omega_i = \prod_{j\neq i} \sigma_j, \quad \varphi_i \equiv \omega_i^{-1} \bmod \sigma_i, \quad \Lambda(x) = \kappa_{tu} \sum_{i=1}^{n} \omega_i\varphi_i + \prod_{i=1}^{n}(x - \sigma_i) = \sum_{k=0}^{n} a_k x^k \tag{3.6}$$

$$\text{GD6:} \quad \kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i \tag{3.7}$$

CHAPTER 4

Critical Analysis of the Base Scheme

Each finding states *what* fails, *where* in the paper, *why*, and the *impact* on the paper's claims. Where possible we confirmed the finding with a script; such cases are marked *verified*.

4.1 Finding 1: the scheme is not infrastructure-less

Authentication requires a live satellite link to the TA

Finding 1. In Step GA1 the TUAV uploads every request to the TA. In Step GA2 the TA computes $\sigma_i, \psi_i, \mu_i, \delta_i$, which include all pairings and need the secret s_i. The verification equation (3.5) of Step GA3 has no inputs other than these values. Nothing in the paper gives a local fallback.

Why it matters. The paper's own system model says the TA is reached through "space networks" (satellite), because the terrestrial backhaul is destroyed. In the paper's scenario this link is shared, congested and possibly jammed. If A is the probability that the TA link is usable, the probability that an authentication round succeeds is exactly A. Its latency also includes one satellite round trip (typically about 30–60 ms for LEO and about 500–600 ms for GEO).
Impact. When the link is lost, authentication does not slow down, it *stops*. The dependency on infrastructure was not removed; it was moved from a roadside pole to a satellite link that nobody in the field controls.

4.2 Finding 2: the TA can impersonate every UAV

Look again at Step US2, equation (3.3). The last term uses $h_1(r_i)$, and Step US1 defined $\xi_i = h_1(r_i)$, which is sent in clear in Step US3. Substituting gives

$$\Gamma_i = \left[s_i\, h_3(ID_i, ts_2, s_i)\, R_{tu} - h_1(s_i \xi_i)\right] P - \upsilon_i\, \xi_i\, Q_{tu}. \tag{4.1}$$

Every quantity on the right is either the TA-issued secret s_i or public ($ts_2, \xi_i, \upsilon_i, R_{tu}, Q_{tu}$; ID_i needs id_i, which the TA also holds). The UAV's own random value r_i enters only through ξ_i, and ξ_i is public.

Key escrow and no non-repudiation (Table II claims F4 and F10 do not hold)

Finding 2. The TA (or anyone who breaches it) can produce a valid request for any UAV i in any session: choose r_i, compute ξ_i, ID_i, υ_i, then compute Γ_i by (4.1). The request satisfies (3.5) for the same algebraic reason an honest one does.

Why. In real certificateless cryptography [6] a user's full private key combines a KGC-issued *partial* key with a *secret value* the user picks and never reveals. Here the user's part (r_i) is effectively published, so only the TA-issued part remains. The scheme is effectively a shared-key scheme with the TA, presented in pairing notation.
Confirmed. Our implementation of the base scheme (`cutcord/base.py`) accepts a request that the TA built without the UAV (experiment E7), and ProVerif finds the same attack on a symbolic model of the credential (Chapter 13).
Consequences. (i) *Key escrow* (F4 ✗): the TA can sign for any UAV. It also computes every σ_i in GA2, so it can read the group key κ_{tu}. (ii) *Non-repudiation* (F10 ✗): verification needs

$\mu_i = \hat{e}(h_3(\cdot)P, s_iP)$, which only the holder of s_i can compute. Any credential could therefore have been produced by the TA, and a UAV can always deny it. (iii) The pairings are not even necessary: the TA could check Γ_i by recomputing (4.1) with two scalar multiplications.

4.3 Finding 3: no forward secrecy

One leaked s_i exposes every past group key (Table II claim F7 does not hold)

Finding 3. The decrypting key $\sigma_i = h_1(s_i\xi_i)P$ is a deterministic function of the long-term secret s_i and the public ξ_i. An adversary that records traffic and later learns s_i (for example by capturing the UAV) recomputes σ_i for every recorded session and then $\kappa_{tu} = \Lambda(\sigma_i) \bmod \sigma_i$ from the recorded coefficients.

Why. Forward secrecy needs an *ephemeral* secret that is erased after the session, for example a Diffie–Hellman exponent. The scheme has none.

4.4 Finding 4: the CRT group key leaks, and revocation fails

(a) The construction is not well defined. Step GA2 defines $\sigma_i = h_1(s_i\xi_i)P$, an *elliptic-curve point*. Step GD1 then multiplies points ($\omega_i = \prod_{j\neq i}\sigma_j$), inverts modulo a point ($\varphi_i \equiv \omega_i^{-1} \bmod \sigma_i$), and Step GD6 reduces modulo a point. None of these operations is defined for points. Under any reading that makes them work (for example σ_i replaced by an integer derived from the point), the paper still omits three conditions that correctness requires: the σ_i must be pairwise coprime, $\kappa_{tu} < \min_i \sigma_i$, and (3.6) must be evaluated over the integers, not modulo q.

(b) The polynomial reveals everyone's key.

Lemma 1. *Let $C = \kappa_{tu}\sum_i \omega_i\varphi_i$. Then for every member j, $\Lambda(\sigma_j) = C$ and $\Lambda(x) - \Lambda(\sigma_j) = \prod_{i=1}^{n}(x - \sigma_i)$.*

Proof. By (3.6), $\Lambda(x) = C + \prod_i(x - \sigma_i)$ and the product vanishes at $x = \sigma_j$. Subtracting gives the second identity. □

So any member j knows its own σ_j, computes the constant $\Lambda(\sigma_j)$, and obtains the monic integer polynomial $\prod_i(x - \sigma_i)$. Its roots, which are the decrypting keys of *all* members, can be found in polynomial time (integer root finding). Note also that the coefficients $a_1, \ldots, a_n$ are exactly the coefficients of $\prod_i(x - \sigma_i)$; only a_0 is masked.

A revoked UAV still obtains the new group key (verified)

Finding 4. In Step DU1 the TUAV builds $\Lambda^*(x) = \prod_{i\in S'}(x - \sigma_i) + \kappa^*_{tu}\sum_{i\in S'}\omega_i\varphi_i$ over the remaining set S', and *reuses* the remaining members' old σ_i (the paper presents this as an advantage). A member k that is removed had already learned every σ_j through Lemma 1. It picks any remaining $j \in S'$ and computes

$$\kappa^*_{tu} = \Lambda^*(\sigma_j) \bmod \sigma_j.$$

Verification. Our script `scripts/crt_revocation_attack.py` builds GD1–GD2 for $n = 12$ members with 160-bit prime moduli, recovers all twelve σ_i from member 0's view, revokes two members, rebuilds Λ^* by DU1, and checks the attack:

```
correctness holds for all 12 members
(A) insider recovered all sigma_i:  True
   revoked member's own sigma gives kappa*?  False
(B) revoked member, using a stolen sigma, gets kappa*:  True
```

Impact. Group forward secrecy after revocation fails, and so does the claim of Theorem 5 that "removed UAVs are locked out". A UAV that joins once and is then evicted keeps reading the group traffic.

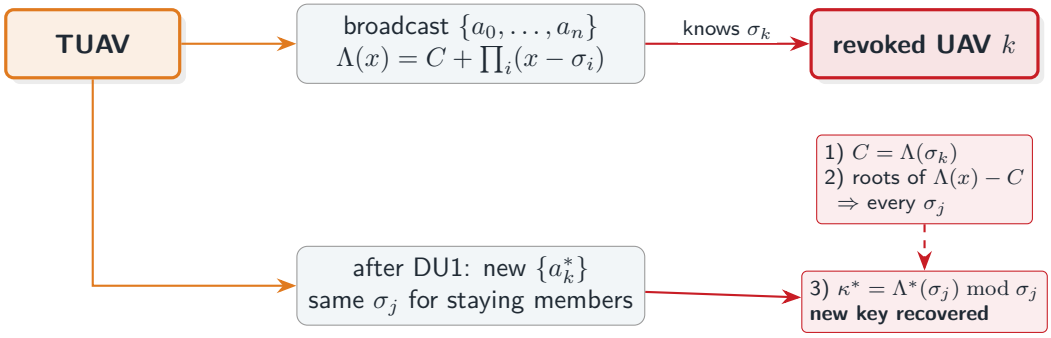


Figure 4.1: Revocation break (Finding 4): an insider learns every decrypting key from the public coefficients and later uses a staying member's key to open the updated polynomial.

(c) The real size of the broadcast. The coefficients a_k are integers whose length grows with n (they are products of up to n 160-bit moduli). Table V of [1] counts only the request, $104n$ bytes. We computed the exact size of $\{a_k\}$ for random 160-bit prime moduli (Table 4.1). Because Step GD3 sends the acknowledgement "one by one to each vehicle", the total traffic is n times larger again: $O(n^2)$ bits per ACK and $O(n^3)$ in total.

Table 4.1: Measured size of the CRT coefficient set (`scripts/compute_results.py`).

n	per ACK	all n ACKs	paper's Table V ($104n$)
20	4.1 KiB	83.0 KiB	2.0 KiB
60	36.0 KiB	2.11 MiB	6.1 KiB
120	142.7 KiB	16.72 MiB	12.2 KiB

4.5 Finding 5: the unforgeability proof is empty

Theorem 1 reduces to a problem that is not hard

Finding 5. The proof of Theorem 1 embeds the CDH instance $(aP, bP) = (s_{tu}P,\ (\sum_i \upsilon_i \xi_i)P)$. But υ_i and ξ_i are sent in clear in every request, so $b = \sum_i \upsilon_i \xi_i$ is public and $abP = b \cdot (s_{tu}P)$ can be computed directly. Solving this "instance" is not hard, so the reduction proves nothing.

Further gaps. (i) The reduction is built around the TUAV's secret s_{tu}, whereas forging a *UAV's* credential should depend on that UAV's secret. (ii) Theorems 2–5 are argued only in words. (iii) The batch check (3.5) uses no random small exponents [11]. Because $\prod_i \psi_i = \hat{e}(\sum_i(\Gamma_i + \sigma_i), P)$, replacing Γ_1, Γ_2 by $\Gamma_1 + \Delta$, $\Gamma_2 - \Delta$ leaves the batch valid although both individual credentials are now invalid. (iv) Step GA4 has no else-branch, so a single bad request rejects the whole batch and the culprit cannot be identified.

4.6 Finding 6: the cost accounting leaves out the expensive part

The reported 151 ms excludes the TA's pairings (and the satellite delay)

Finding 6. Section VI-A states that "the computation overhead on the TUAV side is considered" for GA. However, Step GA2 performs, per UAV, three pairings $(\psi_i, \mu_i, \delta_i)$, three scalar multiplications in $\mathbb{G}_1$ and one point addition:

$$T_{\mathsf{TA}} = 3T_P + 3T_{SM\text{-}P} + T_{PA\text{-}P} + 2T_{SH} = 29.7414 \text{ ms per UAV.}$$

In addition, Γ_i is costed in the 160-bit group $\mathbb{G}$ ($T_{SM\text{-}E}$), but GA2 pairs it, so it must live in the pairing group $\mathbb{G}_1$. Correcting both (constants from [1]):

Quantity	reported in [1]	honest accounting	ratio
UAV signing (US)	1.8203 ms	5.3970 ms ($\Gamma_i \in \mathbb{G}_1$)	3.0×
Single verification (SA)	2.1578 ms	31.8992 ms + RTT	14.8×
Group auth. n=120 (GA)	151.19 ms	3720.16 ms + RTT	24.6×
Request size	104 B	192 B ($\Gamma_i \in \mathbb{G}_1$, 128 B)	1.8×
Satellite traffic per UAV	not reported	192 B up + 512 B down	–
Key ACK, n=120	not reported	142.7 KiB each	–

Impact. With honest accounting the base scheme is the *slowest* of the five schemes it compares (Mei 696 ms, Kumar 1985 ms, Ali 333 ms, Xu 1668 ms at n=120), and it is the only one that needs a wide-area round trip.

4.7 Summary of findings

Table 4.2: Our findings mapped to the base paper's claims.

#	What fails	Where	Claim affected	Verified
1	Authentication needs a live satellite link to the TA	GA1–GA3	"infrastructure-less"	inspection, E1
2	TA can forge any UAV; verification needs s_i	US2, GA2	F4, F10	code (E7) + ProVerif
3	Past group keys recoverable from s_i	GA2, GD6	F7	algebraic
4	Every member learns all σ_i; revoked UAV gets new key	GD2, DU1	F3, F5, Thm 5	code (E7, tests)
5	Theorem 1's reduction is empty; batch check unsound; no failure branch	Thm 1, GA3–4	F1, F8	code (E3, E7)
6	TA pairings and satellite RTT omitted; Γ_i in wrong group	Sec. VI	Tables III–V	measured (E2, E4)

Key idea

All six findings share one root cause: **the TA is inside every operation**. It holds every secret, performs every check, and is reachable only over the link the disaster degrades. Removing the TA from the field is therefore the central design goal of our framework.

CHAPTER 5

System Model, Threat Model and Design Goals

5.1 System model

We keep the base paper's scenario: RSUs are destroyed, and an ECRV with its TUAV is deployed. We extend it in two directions: vehicles become first-class participants, and *several* TUAVs cover neighbouring cells (Figure 5.1).

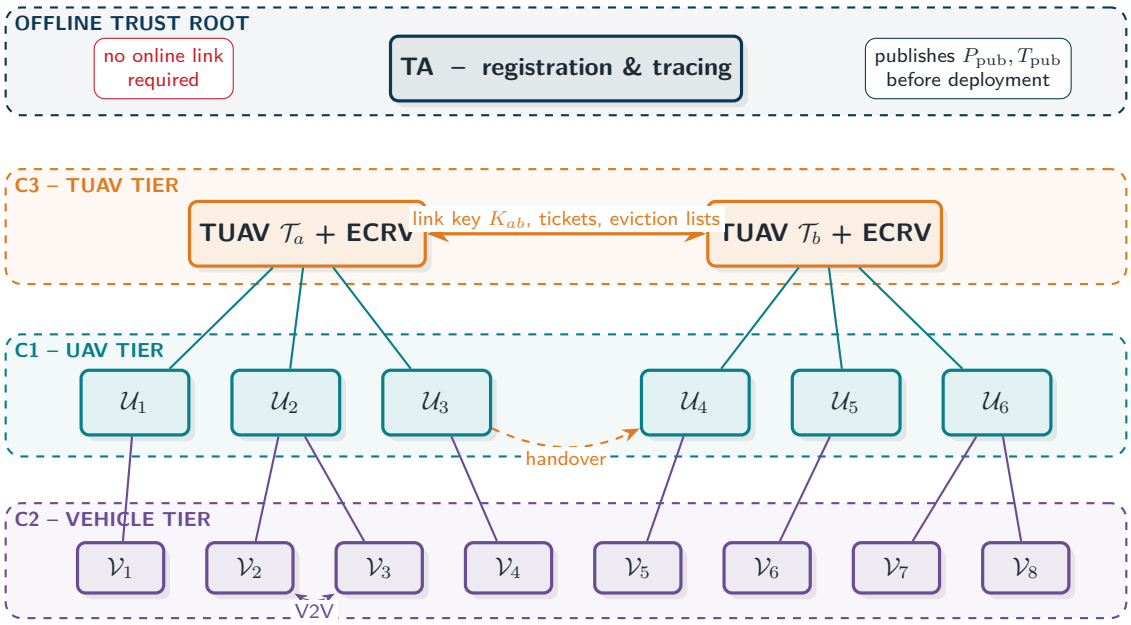


Figure 5.1: The three-tier framework. The TA (top) acts only before deployment and for later tracing; all field operations (C1–C3) run without it.

- **Trusted Authority TA**: generates system parameters, registers every entity *before* deployment, and can later trace a pseudonym to a real identity. It is *not* assumed reachable during operation.
- **TUAV $\mathcal{T}_a$**: the cell controller (powered by its ECRV). It verifies UAVs and vehicles, distributes keys, and talks to neighbouring TUAVs. TUAVs have fixed public identities ID_a; they are infrastructure, not users, so they need no anonymity.
- **UAVs $\mathcal{U}_i$**: battery-powered and resource-limited. They form the aerial relay layer.
- **Vehicles $\mathcal{V}_j$**: hold the same kind of credential as UAVs, reach a TUAV through a UAV or directly, and communicate V2V.

5.2 Threat model

- **Network adversary** (Dolev–Yao [22]): reads, drops, replays, modifies and injects messages on every wireless link.
- **Type-I adversary** [6]: an outsider who may *replace* any entity's public key but does not know the TA master key s.
- **Type-II adversary**: an honest-but-curious (or breached) TA that knows s and all partial keys but not an entity's secret value x. This is the adversary under which the base scheme fails (Finding 2). Following Girault [7], our goal against it is *level 2*: the TA must not be able

to sign under an entity's *registered* key. Like every certificateless scheme, we cannot stop a TA from enrolling a *new* key for an identity (key replacement or phantom enrolment), but such cheating leaves two keys for one identity and is therefore detectable.

- **Insiders**: compromised or revoked UAVs and vehicles that know their own keys and every group key they ever legitimately received.
- **Batch poisoner**: injects requests with invalid signatures so that batches fail (denial of service).

Trust assumptions. TUAVs are trusted to run the protocol honestly; a *captured* TUAV is discussed as a limitation (Chapter 17). Clocks are loosely synchronised (GNSS) for timestamp checks. Hash functions are modelled as random oracles.

5.3 Design goals

Goal	Meaning	Fixes
G1 Authority independence	No field operation (C1–C3) waits for the TA	Finding 1
G2 Escrow-freeness	The TA cannot sign under, or derive the session keys of, any registered key (Girault level 2)	Finding 2
G3 Unforgeability & non-repudiation	Publicly verifiable signatures under ECDLP, with a valid proof sketch	Findings 2, 5
G4 Conditional privacy	Pseudonyms that cannot be linked across epochs; only the TA can trace them, offline	(kept)
G5 Key secrecy	Forward secrecy against long-term key loss; leavers cannot read new keys, joiners cannot read old ones	Findings 3, 4
G6 Robust batching	Sound batch test; bounded cost when a batch fails	Finding 5
G7 Coverage	Vehicles included; seamless handover between TUAVs	new
G8 Efficiency	Comparable cost to the base scheme under the same constants, linear traffic	Finding 6

CHAPTER 6

Preliminaries and Notation

Elliptic-curve group. $\mathbb{G}$ is a cyclic group of prime order q on an elliptic curve with generator P. We need no pairing.

Definition 1 (ECDLP)**.** Given (P, aP) with $a \in \mathbb{Z}_q^*$, computing a is infeasible for any probabilistic polynomial-time (PPT) algorithm.

Definition 2 (CDH)**.** Given (P, aP, bP), computing abP is infeasible for any PPT algorithm.

Schnorr-type signature. A signer with key z and public key $Z = zP$ picks r, sets $R = rP$, $c = H(m, R, \dots)$, $v = r + cz$; the verifier checks $vP = R + cZ$. Unforgeability reduces to ECDLP in the random-oracle model via the forking lemma [9, 10].

Certificateless keys [6]. A full private key combines a KGC-issued *partial key* with a *secret value* chosen by the user, so neither the KGC alone nor a key-replacing outsider can sign. Pairing-free certificateless signatures of this kind are well studied; ECLAS [8] (reference [34] of the base paper) is one example.

Small-exponent batch test [11]. To check n equations $v_i P = R_i + c_i pk_i$ at once, the verifier draws random $\lambda_i \in [1, 2^{\ell_\lambda}]$ and checks one weighted sum. An invalid batch passes with probability at most $2^{-\ell_\lambda}$.

Symmetric tools. KDF is HKDF [20]; $(\mathsf{Enc}, \mathsf{Dec})$ is an authenticated encryption scheme with associated data (AES-128-GCM [21]); MAC is HMAC-SHA-256. $\mathsf{Enc}(K, m; AD)$ encrypts m and authenticates both m and the associated data AD.

Table 6.1: Notation of the proposed framework (our symbols).

Symbol	Meaning
$\mathsf{TA},\ \mathcal{T}_a,\ \mathcal{U}_i,\ \mathcal{V}_j,\ \mathcal{E},\ \mathcal{M}$	Trusted Authority; TUAV of cell a; UAV i; vehicle j; any entity; a moving member
$\mathbb{G},\ q,\ P$	elliptic-curve group, its prime order, generator
$s,\ P_{\text{pub}} = sP$	TA master key (signs partial keys) and its public key
$t,\ T_{\text{pub}} = tP$	TA tracing key (opens pseudonyms) and its public key
$RID_{\mathcal{E}}$	real identity of entity $\mathcal{E}$ (never transmitted)
$ep,\ PID_{\mathcal{E}} = (PID^1_{\mathcal{E}}, PID^2_{\mathcal{E}})$	epoch index; pseudonym of $\mathcal{E}$ valid only in epoch ep
$x_{\mathcal{E}},\ X_{\mathcal{E}} = x_{\mathcal{E}}P$	secret value chosen by $\mathcal{E}$ (TA never sees $x_{\mathcal{E}}$) and its public part
$y_{\mathcal{E}},\ Y_{\mathcal{E}},\ d_{\mathcal{E}}$	TA nonce, its public part $Y_{\mathcal{E}} = y_{\mathcal{E}}P$, and the partial private key
$h_{\mathcal{E}} = H_1(PID_{\mathcal{E}}, ep, X_{\mathcal{E}}, Y_{\mathcal{E}})$	binding hash of the credential
$sk_{\mathcal{E}} = x_{\mathcal{E}} + d_{\mathcal{E}},\ pk_{\mathcal{E}} = X_{\mathcal{E}} + Y_{\mathcal{E}} + h_{\mathcal{E}}P_{\text{pub}}$	full private key; full public key ($sk_{\mathcal{E}}P = pk_{\mathcal{E}}$)
$(R, v),\ c$	signature, $R = rP$, $v = r + c\,sk$; challenge $c = H_2(\cdot)$
$\lambda_i,\ \ell_\lambda$	random batch weights, $\lambda_i \in [1, 2^{\ell_\lambda}]$, $\ell_\lambda = 64$
$e_{\mathcal{E}},\ E_{\mathcal{E}} = e_{\mathcal{E}}P$	ephemeral Diffie–Hellman secret and public value (erased after use)
K_i	pairwise key between $\mathcal{T}_a$ and member i
$GK_a,\ VK_a,\ \nu$	UAV group key, vehicle regional key of cell a, key version
K_{ab}	link key between neighbouring TUAVs $\mathcal{T}_a, \mathcal{T}_b$
$Tk_{\mathcal{M}},\ hk_{\mathcal{M}},\ N_{\mathcal{M}},\ exp$	handover ticket, handover key, ticket nonce, ticket expiry
$ts,\ \Delta t$	timestamp, acceptance window
$\mathcal{L}$	eviction/revocation list (hashes of banned pseudonyms)
$H_0 : \mathbb{G} \times \{0,1\}^* \to \{0,1\}^{\lvert RID\rvert},\ H_1, H_2 : \{0,1\}^* \to \mathbb{Z}_q^*,\ H_3$	hash functions (H_3: SHA-256, used for list indices)
$\Vert$	concatenation

CHAPTER 7

Proposed Framework: Setup and Offline Registration

The framework has five phases (Figure 7.1). Only Phase 0 involves the TA; Phases 1–3 run in the field without it; Phase 4 is optional and happens whenever the TA becomes reachable, without blocking anything.

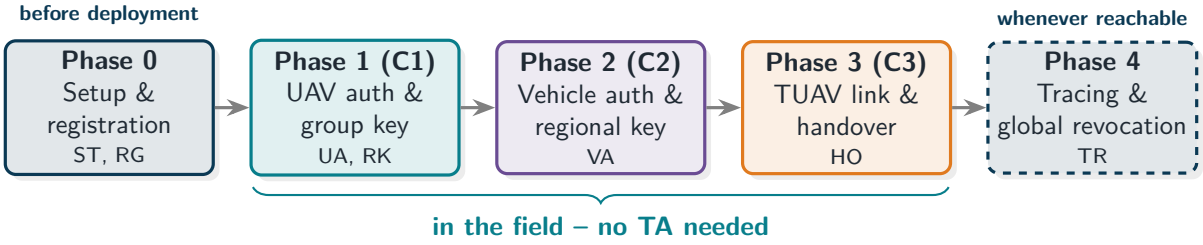


Figure 7.1: Phases of the framework and their step labels.

7.1 System setup (ST)

ST1–ST2 (TA, once)

ST1. TA chooses $(\mathbb{G}, q, P)$, the master key $s \in \mathbb{Z}_q^*$ and the tracing key $t \in \mathbb{Z}_q^*$, and sets $P_{\text{pub}} = sP$, $T_{\text{pub}} = tP$.
ST2. TA publishes $params = \langle \mathbb{G}, q, P, P_{\text{pub}}, T_{\text{pub}}, H_0, H_1, H_2, H_3, \mathsf{KDF}, \mathsf{Enc}, \mathsf{MAC} \rangle$ and keeps (s, t) secret.

Why it is designed this way

Two separate TA keys give *separation of duties*. s certifies keys and t opens pseudonyms, so the two can be held by different offices (for example, the network operator and the law-enforcement agency). Leaking one does not give the power of the other.

7.2 Entity registration (RG)

Every UAV, vehicle and TUAV registers once, over a secure channel, before deployment. A mission is divided into m *epochs* (for example 15 minutes each), and an entity receives one pseudonym credential per epoch.

RG1–RG4 (entity $\mathcal{E} \leftrightarrow$ TA, offline)

RG1. For each epoch $ep = 1..m$, $\mathcal{E}$ picks a secret value $x_{\mathcal{E}}^{(ep)} \in \mathbb{Z}_q^*$ and computes $X_{\mathcal{E}}^{(ep)} = x_{\mathcal{E}}^{(ep)} P$. It sends $RID_{\mathcal{E}}$ and $\{X_{\mathcal{E}}^{(ep)}\}$ to TA, together with a Schnorr signature under each $x_{\mathcal{E}}^{(ep)}$ on $RID_{\mathcal{E}} \,\|\, X_{\mathcal{E}}^{(ep)}$ (proof of possession). **The values $x_{\mathcal{E}}^{(ep)}$ never leave the entity.**
RG2. For each ep, TA picks $\rho, y \in \mathbb{Z}_q^*$ and computes

$$PID^1 = \rho P, \qquad PID^2 = RID_{\mathcal{E}} \oplus H_0(\rho\, T_{\text{pub}} \,\|\, ep), \qquad Y = yP,$$

$$h = H_1(PID, ep, X_{\mathcal{E}}^{(ep)}, Y), \qquad d = y + s\,h \bmod q.$$

TA stores $(RID_{\mathcal{E}}, \{PID^{(ep)}\})$ and the signed RG1 record, then erases ρ, y.

RG3. TA $\to \mathcal{E}$: $\{(PID^{(ep)}, Y^{(ep)}, d^{(ep)})\}_{ep=1}^{m}$.

RG4. $\mathcal{E}$ checks $d^{(ep)}P \stackrel{?}{=} Y^{(ep)} + h^{(ep)}P_{\text{pub}}$ for every ep, and stores the credentials.

A TUAV registers in the same way with its fixed public identity ID_a in place of a pseudonym and a single credential (X_a, Y_a, d_a), $h_a = H_1(ID_a, 0, X_a, Y_a)$.

Correctness of the keys. With $sk = x + d$ and $pk = X + Y + hP_{\text{pub}}$: $sk \cdot P = xP + (y + sh)P = X + Y + hP_{\text{pub}} = pk$. *From here on, inside one epoch we write $x_{\mathcal{E}}, X_{\mathcal{E}}, PID_{\mathcal{E}}, Y_{\mathcal{E}}, d_{\mathcal{E}}, h_{\mathcal{E}}$ without the superscript.*

What this step does

Each entity ends up with a full key pair $(sk_{\mathcal{E}}, pk_{\mathcal{E}})$ per epoch. The TA certified half of it $(d_{\mathcal{E}})$ but never saw the other half $(x_{\mathcal{E}})$. Anyone holding only P_{pub} can check that $pk_{\mathcal{E}}$ was certified, because $pk_{\mathcal{E}}$ contains $h_{\mathcal{E}}P_{\text{pub}}$.

Why it is designed this way

- **Escrow-free (fixes Finding 2).** Signing needs $sk = x + d$. The TA knows d but not x, so it cannot sign for $\mathcal{E}$.
- **No certificates.** The binding hash $h = H_1(PID, ep, X, Y)$ ties the TA's signature to this exact X. An outsider who replaces X changes h, and then needs a new d, which requires s.
- **Fresh $x^{(ep)}$ per epoch.** If X were the same in every epoch it would link the pseudonyms together. Per-epoch values keep epochs unlinkable.
- **Pseudonym = ElGamal-style encryption of RID under T_{pub}.** Only the holder of t can open it ($\rho T_{\text{pub}} = t\,PID^1$).
- **Proof of possession and the signed record.** A TA cannot later claim that an entity owns a key it does not, which supports non-repudiation in disputes.

CHAPTER 8

C1: Autonomous UAV Authentication and Group Key Management

Key idea

The TUAV verifies all requesting UAVs *itself*, using only P_{pub}, in one small-exponent batch. Each verified UAV receives the group key encrypted under its own fresh Diffie–Hellman key. No satellite link, no pairings, no CRT polynomial.

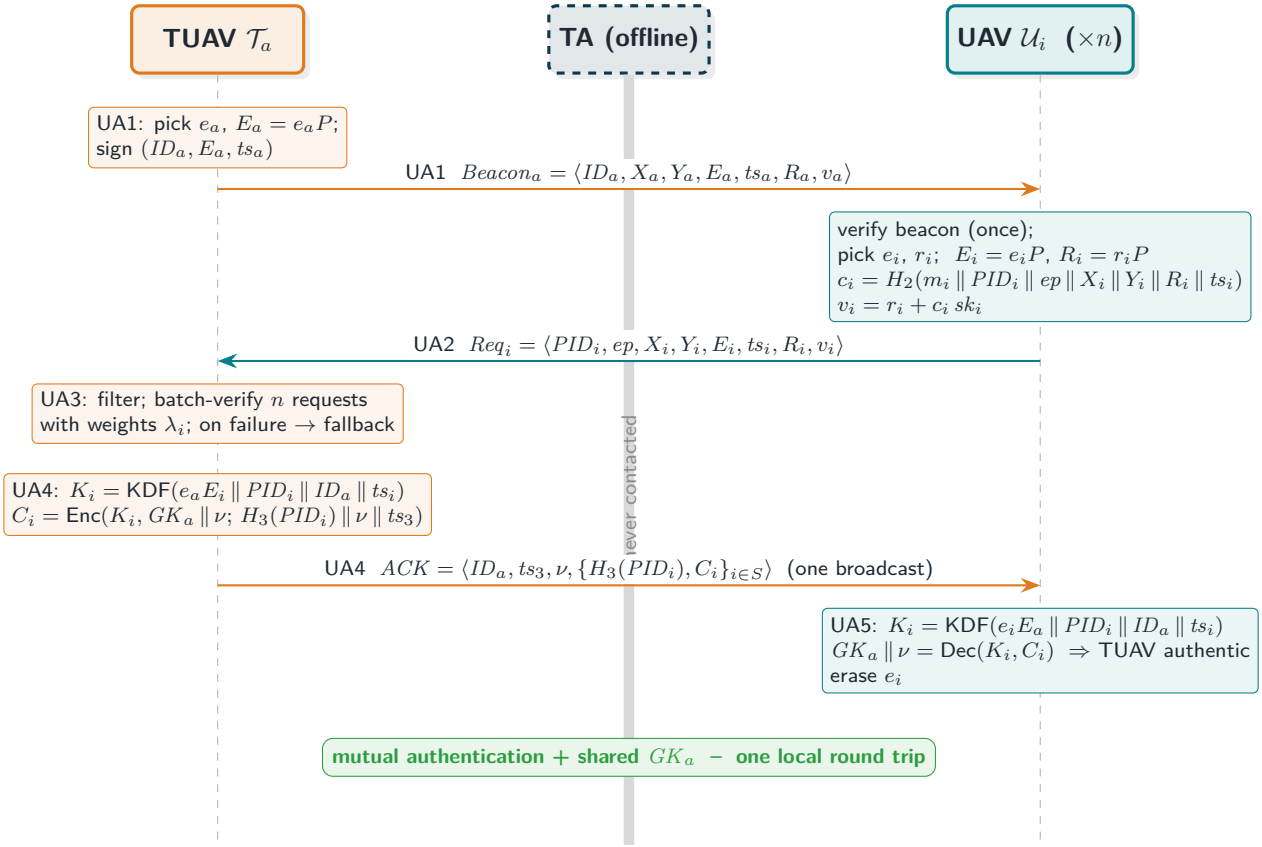


Figure 8.1: C1 message flow. Compare with Figure 3.1: the satellite round trip and the TA's pairings are gone.

8.1 Steps

Let $m_i = (\texttt{"join"} \| ID_a \| E_a \| E_i)$ be the content a UAV signs; the TUAV can rebuild m_i from the request and its own beacon.

UA1 Beacon (TUAV, periodic)

$\mathcal{T}_a$ picks a fresh $e_a \in \mathbb{Z}_q^*$, sets $E_a = e_a P$, signs $(ID_a \| E_a \| ts_a)$ with sk_a to obtain (R_a, v_a), and broadcasts $Beacon_a = \langle ID_a, X_a, Y_a, E_a, ts_a, R_a, v_a \rangle$. A new e_a is drawn every beacon period, and the old one is erased when the period ends.

UA2 Signed join request (each UAV)

$\mathcal{U}_i$ checks $|ts_a - ts_{\text{now}}| \le \Delta t$ and verifies the beacon once: $v_a P \stackrel{?}{=} R_a + c_a\, pk_a$. It picks $e_i, r_i \in \mathbb{Z}_q^*$, computes $E_i = e_i P$, $R_i = r_i P$,

$$c_i = H_2(m_i \,\|\, PID_i \,\|\, ep \,\|\, X_i \,\|\, Y_i \,\|\, R_i \,\|\, ts_i), \qquad v_i = r_i + c_i\, sk_i \bmod q,$$

and sends $Req_i = \langle PID_i, ep, X_i, Y_i, E_i, ts_i, R_i, v_i \rangle$. **Cost: two scalar multiplications** (plus one beacon verification per beacon period).

UA3 Local batch verification with bounded fallback (TUAV)

Filter. Collect the requests of one window. Drop any request whose timestamp is stale, whose ep is not current, whose $H_3(PID_i) \in \mathcal{L}$, or whose PID_i already sent a request in this window.
Batch test. Compute $h_i = H_1(PID_i, ep, X_i, Y_i)$ and c_i, draw $\lambda_i \in [1, 2^{64}]$, and check

$$\Big(\sum_{i=1}^{n} \lambda_i v_i\Big) P \stackrel{?}{=} \sum_{i=1}^{n} \lambda_i R_i + \sum_{i=1}^{n} (\lambda_i c_i)\,(X_i + Y_i) + \Big(\sum_{i=1}^{n} \lambda_i c_i h_i\Big) P_{\text{pub}}. \tag{8.1}$$

If it holds, accept all n. **If it fails**, verify each request individually ($v_i P \stackrel{?}{=} R_i + c_i pk_i$), accept the valid ones, put $H_3(PID_i)$ of each invalid one into the eviction list $\mathcal{L}$ for the rest of the epoch, and process the next window in individual mode (adaptive mode, Section 8.2).

UA4 Group-key delivery (TUAV)

Let S be the accepted set. For each $i \in S$: $K_i = \mathsf{KDF}(e_a E_i \,\|\, PID_i \,\|\, ID_a \,\|\, ts_i)$ and $C_i = \mathsf{Enc}(K_i,\ GK_a \,\|\, \nu;\ H_3(PID_i) \,\|\, \nu \,\|\, ts_3)$. Broadcast *once* $ACK = \langle ID_a, ts_3, \nu, \{H_3(PID_i), C_i\}_{i \in S} \rangle$. $\mathcal{T}_a$ keeps K_i for as long as $\mathcal{U}_i$ is a member.

UA5 Key recovery and TUAV authentication (each UAV)

$\mathcal{U}_i$ finds its entry by $H_3(PID_i)$, computes $K_i = \mathsf{KDF}(e_i E_a \,\|\, PID_i \,\|\, ID_a \,\|\, ts_i)$ and decrypts C_i. If decryption succeeds, then $\mathcal{T}_a$ (the holder of e_a behind the *signed* E_a) is authentic, and $\mathcal{U}_i$ holds GK_a. $\mathcal{U}_i$ erases e_i.

Correctness. (i) For an honest request, $v_i P = r_i P + c_i sk_i P = R_i + c_i pk_i = R_i + c_i(X_i + Y_i) + c_i h_i P_{\text{pub}}$. Multiplying by λ_i and summing gives (8.1). (ii) $e_a E_i = e_a e_i P = e_i E_a$, so both sides derive the same K_i.

Why it is designed this way

- **Verification only needs P_{pub}** (fixes Finding 1). The certificate is "inside" pk_i, so the TUAV needs neither the TA nor any secret.
- **Random weights λ_i** (fixes Finding 5(iii)). Without them, errors in two requests can cancel each other. With 64-bit weights a bad batch passes with probability at most 2^{-64}.
- **Request binds E_a and ID_a.** A request is valid only for this TUAV and this beacon period, so it cannot be replayed into another session or another cell.
- **Ephemeral e_a, e_i** (fixes Finding 3). They are erased after use, so stealing (x_i, d_i) later reveals no past K_i and hence no past GK_a.
- **Per-member ciphertexts instead of a CRT polynomial** (fixes Finding 4). Each C_i is opened only by K_i, and knowing your own K_i says nothing about anyone else's. Size: 68

bytes per member, linear in n.

- **ACK is not signed.** The AEAD tag under K_i already proves that the sender knows e_a; a signature would cost more and add nothing.
- **One pseudonym per epoch.** An evicted UAV cannot simply come back with a fresh pseudonym in the same epoch.

8.2 When a batch fails: bounded fallback

The classical answer to a failed batch is split-and-retest (binary search for bad signatures [14]). We evaluated it for our scheme and found it does *not* pay off. A pairing-free batch is only about 2.8× cheaper per signature than individual verification (0.98 ms versus 2.74 ms), so re-testing halves costs more than simply checking everyone (Table 8.1).

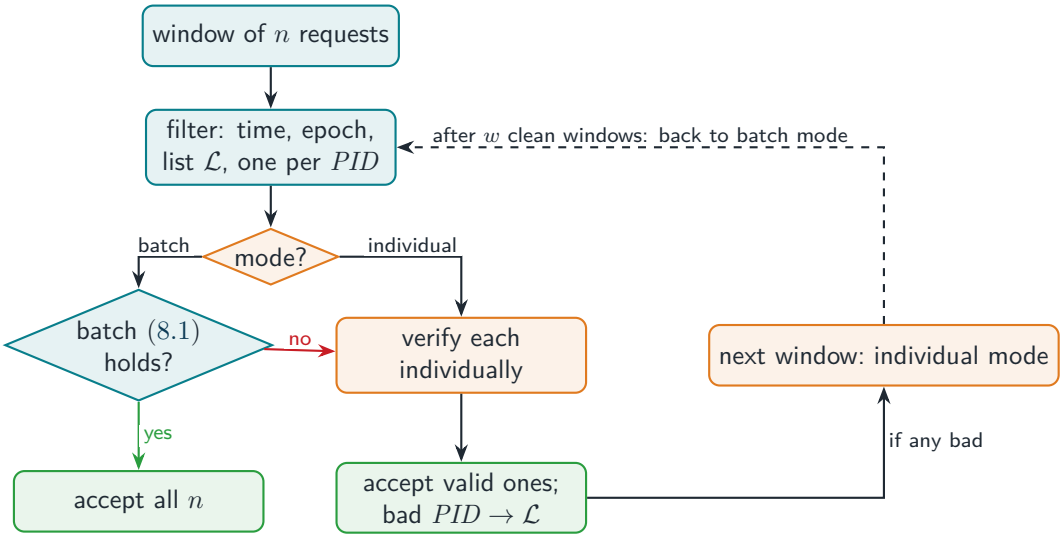


Figure 8.2: Adaptive verification rule of Step UA3.

Proposition 1 (Bounded cost under batch poisoning)**.** *For any number $d \geq 1$ of invalid requests in a window of n, Step UA3 costs at most $T_{BV}(n) + n\,T_{SV}$, and every valid request is accepted. In individual mode it costs exactly $n\,T_{SV}$. With the constants of [1], $n = 120$: clean 119.9 ms, worst case 448.6 ms (3.7×), individual mode 328.8 ms. In the base scheme a single invalid request makes* all *n fail (goodput 0).*

Table 8.1: Verification time at the TUAV for $n = 120$ with d invalid requests (average of 200 random placements; `compute_results.py`).

bad (%)	d	split-and-retest (ms)	our fallback (ms)	individual (ms)	our goodput (%)	base goodput (%)
0	0	119.9	119.9	328.8	100	100
2	2	464	448.6	328.8	98	0
4	5	630.7	448.6	328.8	96	0
6	7	698.3	448.6	328.8	94	0
8	10	762.4	448.6	328.8	92	0
10	12	795.8	448.6	328.8	90	0
20	24	915.9	448.6	328.8	80	0

8.3 Rekeying on leave, eviction and join (RK)

RK1–RK2 Group-key update (TUAV)

RK1. Let S' be the new member set: leavers and evicted members are removed, and joiners are added after their own UA2–UA4. $\mathcal{T}_a$ draws a *fresh* GK'_a, sets $\nu \leftarrow \nu + 1$, and broadcasts $\langle ID_a, ts, \nu, \{H_3(PID_i), \mathsf{Enc}(K_i, GK'_a \,\|\, \nu;\ H_3(PID_i) \,\|\, \nu \,\|\, ts)\}_{i \in S'} \rangle$.
RK2. Each $\mathcal{U}_i \in S'$ decrypts its entry and switches to GK'_a.

Why it is designed this way

A removed member has no K_j of anyone else, so it cannot open any entry. That is the property the CRT construction lost (Finding 4). A joiner receives only the fresh GK'_a, so it cannot read traffic sent before it joined. A version ν in the associated data stops an old ciphertext being replayed as a new key.

Optional logarithmic rekeying (LKH). For very large swarms the TUAV can arrange members as leaves of a binary key tree [15], using K_i as leaf keys. A single leave then costs $2\lceil \log_2 n \rceil - 1$ ciphertexts (13 for $n = 120$) instead of $n - 1$. Section 15 reports both variants.

CHAPTER 9

C2: The Vehicle Tier

The base paper secures only the UAV–TUAV link and states that vehicle authentication is "well studied" and assumed to be handled elsewhere. Yet the vehicles are why the network exists. Existing vehicle schemes assume an RSU or an online TA, neither of which is available here. C2 brings vehicles under the *same* offline trust root.

Key idea

Vehicles carry the same certificateless credential as UAVs. Authenticated UAVs act as *relays and filters*: they carry vehicles' requests to the TUAV, which batch-verifies them and returns a regional key for V2V traffic. The UAVs cannot read or forge anything.

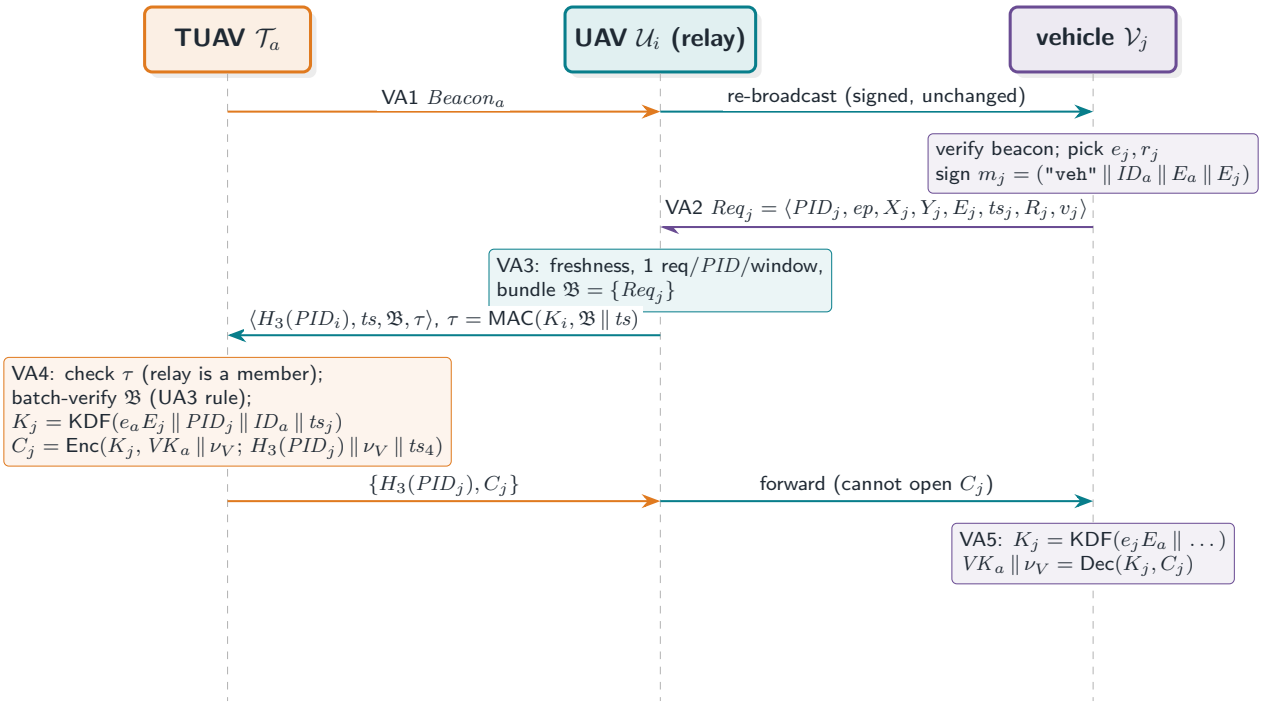


Figure 9.1: C2: vehicle authentication relayed by an authenticated UAV.

9.1 Steps

VA1–VA5 Vehicle admission

VA1. UAVs re-broadcast the TUAV's signed beacon so that vehicles out of the TUAV's range receive it. A relay cannot alter it.

VA2. $\mathcal{V}_j$ verifies the beacon, picks e_j, r_j, and signs $m_j = (\texttt{"veh"} \| ID_a \| E_a \| E_j)$ exactly as in UA2, giving Req_j. It sends the request to a UAV in range, or directly to $\mathcal{T}_a$ if it is in range.

VA3. The relay $\mathcal{U}_i$ checks freshness, allows at most one request per PID_j per window, collects a bundle $\mathfrak{B}$ and sends $\langle H_3(PID_i), ts, \mathfrak{B}, \tau = \mathsf{MAC}(K_i, \mathfrak{B} \| ts) \rangle$ to $\mathcal{T}_a$.

VA4. $\mathcal{T}_a$ discards the bundle unless τ verifies under the key of a *current* member. It then applies the UA3 rule to $\mathfrak{B}$, derives K_j for each valid vehicle, and returns $C_j =$

$\mathsf{Enc}(K_j, VK_a \,\|\, \nu_V;\ H_3(PID_j) \,\|\, \nu_V \,\|\, ts_4)$ through $\mathcal{U}_i$.
VA5. $\mathcal{V}_j$ derives K_j, decrypts VK_a and erases e_j. A successful decryption authenticates $\mathcal{T}_a$.

9.2 V2V communication under the regional key

Vehicles send two classes of message:

- **Routine beacons** (position, speed; about 10 per second): $\langle m, PID_j, ts, \mathsf{MAC}(VK_a, m \,\|\, PID_j \,\|\, ts)\rangle$. Cost: one HMAC.
- **Safety-critical events** (accident, road blocked): $\langle m, PID_j, ep, X_j, Y_j, ts, R, v\rangle$ with a certificateless signature as in UA2. Receivers verify bursts of events with the batch test (8.1).

Why it is designed this way

- **Same credential, same root.** One registration serves both UAVs and vehicles, so a vehicle can be verified anywhere using only P_{pub}.
- **UAVs relay, the TUAV verifies.** Battery-limited UAVs do no public-key verification for vehicles. The TUAV, which has a cable power supply, does the batch work, where batching pays off most (hundreds of vehicles).
- **Relay MAC as a flood filter.** Only bundles from current members reach the verifier, and each relay rate-limits per pseudonym. An outsider flooding forged requests is stopped at the first hop, which addresses the denial-of-service gap of Finding 5(iv) at the vehicle scale.
- **Separate keys VK_a and GK_a.** A captured vehicle must not obtain the UAV control-channel key (privilege separation).
- **Two message classes.** A MAC under a shared key is fast but only proves "some member sent this". Safety-critical events need non-repudiation (accountability after an accident), so they are signed.

Honest limitation

Any holder of VK_a can forge a *routine* beacon that appears to come from another pseudonym. We accept this for routine traffic and require signatures for events that matter. A relay UAV can drop bundles (denial of service) but cannot forge, read or alter them undetected.

CHAPTER 10

C3: TUAV-to-TUAV Handover

One TUAV covers a limited area, so a real disaster zone needs several, and UAVs and vehicles move between cells. Re-running the full C1/C2 admission at every cell border costs signature generation and verification on both sides and interrupts service. The base paper considers a single TUAV only.

Key idea

Neighbouring TUAVs share a link key K_{ab}. The old TUAV gives a moving member an encrypted *ticket*. The new TUAV opens the ticket, checks a MAC, runs one ECDH for forward secrecy, and admits the member *without any signature verification and without the TA*.

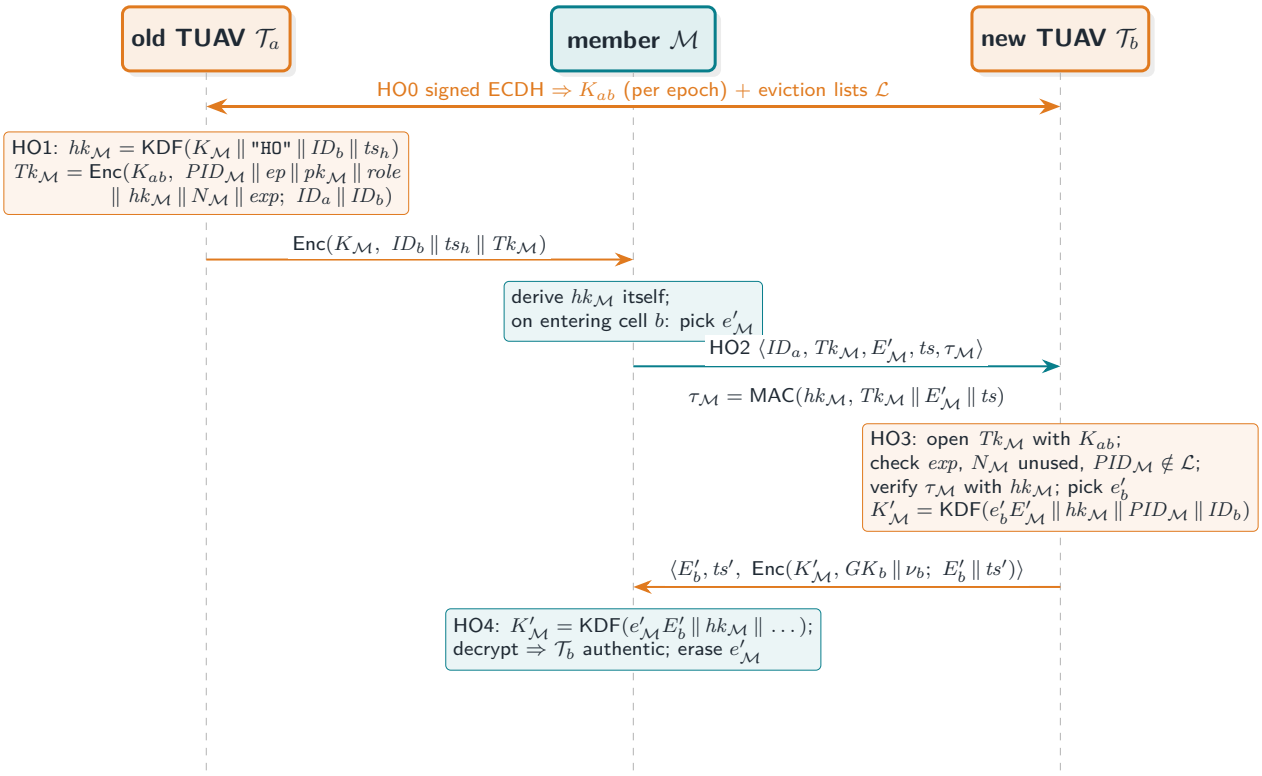


Figure 10.1: C3 handover. Only symmetric operations and one ECDH are needed at the new TUAV.

10.1 Steps

HO0–HO4 Inter-TUAV link and ticket-based handover

HO0. When two TUAVs come into range, and once per epoch after that, they exchange signed ephemeral values (E_{ab}, E_{ba}) using their credentials and derive $K_{ab} = \mathsf{KDF}(e_{ab}E_{ba} \| ID_a \| ID_b \| ts)$. Over K_{ab} they exchange their eviction lists $\mathcal{L}$.

HO1. When member $\mathcal{M}$ (pairwise key $K_{\mathcal{M}}$ with $\mathcal{T}_a$) approaches cell b, $\mathcal{T}_a$ derives $hk_{\mathcal{M}} = \mathsf{KDF}(K_{\mathcal{M}} \| \texttt{"HO"} \| ID_b \| ts_h)$, draws a nonce $N_{\mathcal{M}}$ and an expiry exp, builds the ticket $Tk_{\mathcal{M}} = \mathsf{Enc}(K_{ab},\ PID_{\mathcal{M}} \| ep \| pk_{\mathcal{M}} \| role \| hk_{\mathcal{M}} \| N_{\mathcal{M}} \| exp;\ ID_a \| ID_b)$, and sends it to $\mathcal{M}$ under $K_{\mathcal{M}}$. $\mathcal{M}$ derives the same $hk_{\mathcal{M}}$ from $K_{\mathcal{M}}$.

HO2. In cell b, $\mathcal{M}$ picks $e'_{\mathcal{M}}$ and sends $\langle ID_a, Tk_{\mathcal{M}}, E'_{\mathcal{M}}, ts, \tau_{\mathcal{M}} = \mathsf{MAC}(hk_{\mathcal{M}}, Tk_{\mathcal{M}} \,\|\, E'_{\mathcal{M}} \,\|\, ts)\rangle$ to $\mathcal{T}_b$.

HO3. $\mathcal{T}_b$ decrypts $Tk_{\mathcal{M}}$ with K_{ab} and rejects the request if exp has passed, if $N_{\mathcal{M}}$ was already used (it caches nonces until exp), if $H_3(PID_{\mathcal{M}}) \in \mathcal{L}$, or if $\tau_{\mathcal{M}}$ does not verify under $hk_{\mathcal{M}}$. Otherwise it picks e'_b and sends $\langle E'_b, ts', \mathsf{Enc}(K'_{\mathcal{M}}, GK_b \,\|\, \nu_b;\ E'_b \,\|\, ts')\rangle$ with $K'_{\mathcal{M}} = \mathsf{KDF}(e'_b E'_{\mathcal{M}} \,\|\, hk_{\mathcal{M}} \,\|\, PID_{\mathcal{M}} \,\|\, ID_b)$. For a vehicle ($role = \texttt{veh}$), it sends VK_b instead of GK_b.

HO4. $\mathcal{M}$ derives $K'_{\mathcal{M}}$ and decrypts. Success means $\mathcal{T}_b$ knows $hk_{\mathcal{M}}$, which only a holder of K_{ab} could obtain, so $\mathcal{T}_b$ is authentic. $\mathcal{M}$ and $\mathcal{T}_b$ erase $e'_{\mathcal{M}}, e'_b$. $\mathcal{T}_a$ removes $\mathcal{M}$ at its next rekey.

Why it is designed this way

- **Why a ticket.** The old TUAV already verified $\mathcal{M}$'s signature. The ticket passes that verified fact on, under K_{ab}, so the new TUAV need not repeat the work.
- **Why $hk_{\mathcal{M}}$ and not $K_{\mathcal{M}}$.** $\mathcal{T}_b$ never learns the pairwise key $\mathcal{M}$ used in cell a. If $\mathcal{T}_b$ is later compromised, $\mathcal{M}$'s traffic in cell a stays safe.
- **Why a fresh ECDH.** The new key $K'_{\mathcal{M}}$ mixes a fresh Diffie–Hellman value with $hk_{\mathcal{M}}$, so it has forward secrecy, and an attacker who later learns K_{ab} still cannot compute it.
- **Why nonce, expiry and AD.** A ticket can be used once, only before exp, and only between the two named TUAVs ($ID_a \,\|\, ID_b$ as associated data).
- **Why share $\mathcal{L}$.** An entity evicted in cell a cannot escape by moving to cell b.

CHAPTER 11

The TA's Offline Role: Tracing and Global Revocation

TR1–TR2 (TA, whenever reachable – never on the critical path)

TR1 (tracing). Given a pseudonym from a disputed message, the tracing office computes $RID = PID^2 \oplus H_0(t \cdot PID^1 \,\|\, ep)$, which is correct because $t\,PID^1 = t\rho P = \rho T_{\text{pub}}$. The signed message and the RG1 record bind the real entity to the act (non-repudiation).

TR2 (global revocation). To ban an entity permanently, TA publishes a signed list of $H_3(PID^{(ep)})$ for all of its *remaining* epochs. TUAVs merge the list into $\mathcal{L}$ and propagate it over K_{ab}. Earlier pseudonyms are not published, so the entity's past stays unlinkable.

Why it is designed this way

Revocation becomes *two-speed*. A TUAV evicts a misbehaving pseudonym *immediately and locally* (UA3, RK1, shared via HO0), which covers the rest of the epoch. Permanent revocation waits until the TA is reachable, by any channel at any time, and it never blocks authentication. This is the practical meaning of "infrastructure-less": the infrastructure *helps when present* but is *not required*.

CHAPTER 12

Security Analysis

We give proof *sketches* that follow standard reductions; Chapter 13 complements them with a mechanised ProVerif [23] analysis of the same properties in the symbolic model. Unlike the base paper's Theorem 1 (Finding 5), every reduction below embeds the challenge in a value that is *secret*.

12.1 Authority independence

Proposition 2 (No online TA)**.** *Steps UA1–UA5, RK1–RK2, VA1–VA5 and HO0–HO4 use only (a) the public params, including P_{pub}, (b) keys stored by the participating entities, and (c) messages exchanged between them. No step sends a message to, or waits for a message from, the TA.*

Proof. By inspection of each step. Signature verification needs P_{pub} only (equation (8.1)); key derivation uses ephemeral values and stored pairwise keys; tickets use K_{ab}. □

Consequently the success probability of authentication is independent of the TA-link availability A (Figure 15.2), whereas for the base scheme it equals A.

12.2 Unforgeability, escrow-freeness and non-repudiation

Theorem 12.1 (EUF-CMA)**.** *In the random-oracle model, if ECDLP is hard in $\mathbb{G}$, no PPT Type-I or Type-II adversary can produce a valid signature (R, v) on a new message under the public key of an honest entity, except with negligible probability.*

Proof sketch. Type II (malicious TA, knows s and every d). The reduction $\mathcal{B}$ receives $A = aP$, guesses the target entity, and sets its $X^* = A$; it knows s and can produce all partial keys. It answers signing queries for the target without x^* by programming the oracle: it picks v, c at random, sets $R = vP - c\,pk^*$ and defines $H_2(\cdot) = c$. From a forgery, the forking lemma [10] gives two signatures with the same R and different challenges $c \neq c'$. Then $sk^* = (v - v')/(c - c')$ and $a = x^* = sk^* - d^*$. *Type I (outsider, may replace public keys).* $\mathcal{B}$ sets $P_{\text{pub}} = A$. It issues partial keys without s by choosing d, h at random and setting $Y = dP - hP_{\text{pub}}$ and $H_1(PID, ep, X, Y) = h$. A forgery under a (possibly replaced) key $pk^* = X^* + Y^* + h^* P_{\text{pub}}$ is forked on H_2, which gives $sk^* = \log_P(X^* + Y^*) + h^* a$. Forking again at the H_1 query that fixed h^* gives $sk^{*\prime}$ with $h^{*\prime} \neq h^*$ and the same $X^* + Y^*$, hence $a = (sk^* - sk^{*\prime})/(h^* - h^{*\prime})$. This is the standard double-forking argument for pairing-free certificateless and identity-based Schnorr signatures. □

Theorem 12.2 (Escrow-freeness and non-repudiation)**.** *The TA cannot produce a signature that verifies under a registered entity's public key, so a valid signature (or a valid UA2 request) can be attributed to that entity.*

Proof. This is the Type-II case of Theorem 12.1. The TA's knowledge (s, $d_{\mathcal{E}}$, $PID_{\mathcal{E}}$) does not include $x_{\mathcal{E}}$, and the RG1 proof of possession binds $X_{\mathcal{E}}$ to $RID_{\mathcal{E}}$. (Contrast with Finding 2.) □

Exact scope of escrow-freeness (Girault level 2)

Theorem 12.2 covers signatures under the entity's *registered* key. A malicious TA can still issue a *second* credential for the same pseudonym with a key of its own choosing, or enrol a phantom member. Every certificateless scheme allows this, and ProVerif confirms it (Chapter 13, query Q7). Such cheating is detectable, because two keys then exist for one identity and the honest entity holds its signed RG1 record. Preventing it outright (level 3) would need a binding the TA cannot create, for example a device key certified by the UAV's manufacturer.

12.3 Batch soundness

Theorem 12.3 (Small-exponent test [11])**.** *If at least one request in a batch is invalid, equation* (8.1) *holds with probability at most* $2^{-\ell_\lambda}$ *over the choice of the weights* λ_i*.*

Proof sketch. Write $\Delta_i = v_i P - R_i - c_i pk_i$, with $\Delta_k \neq O$ for some k. Equation (8.1) is $\sum_i \lambda_i \Delta_i = O$. Fix all λ_i with $i \neq k$. Because $\mathbb{G}$ has prime order, at most one value of λ_k satisfies the equation, so the probability is at most $2^{-\ell_\lambda}$. The cancellation attack of Finding 5(iii) is therefore defeated. □

12.4 Mutual authentication and key secrecy

Theorem 12.4 (C1 key secrecy and mutual authentication)**.** *Under CDH in the random-oracle model and the security of the AEAD scheme: (i) only* $\mathcal{T}_a$ *and* $\mathcal{U}_i$ *can compute* K_i*; (ii)* GK_a *is known only to* $\mathcal{T}_a$ *and the current members; (iii)* $\mathcal{T}_a$ *accepts* $\mathcal{U}_i$ *only if* $\mathcal{U}_i$ *signed* E_i *for this beacon; (iv)* $\mathcal{U}_i$ *accepts only the TUAV that signed* E_a*.*

Proof sketch. (i) $K_i = \mathsf{KDF}(e_a e_i P \,\|\, \dots)$. An adversary sees E_a, E_i only, and computing $e_a e_i P$ is CDH. Swapping E_i would break the signature on m_i (Theorem 12.1), and swapping E_a would break the beacon signature. (ii) C_i is AEAD under K_i. An insider $\mathcal{U}_k$ knows only its own K_k, which is independent of every other K_i (unlike Lemma 1). (iii) The request signature covers E_a, ID_a, E_i, ts_i. (iv) Decryption of C_i under K_i succeeds only if the sender knows e_a, and E_a is signed by sk_a. □

Theorem 12.5 (Forward secrecy, and key secrecy under membership change)**.** *(a) Compromise of every long-term key* $(x, d$ *of members,* sk_a *of the TUAV, even* s *and* $t)$ *after a session does not reveal that session's* K_i *or* GK_a*. (b) A member removed in RK1 cannot obtain* GK'_a *(group forward secrecy). (c) A member that joins cannot obtain an earlier* GK_a *(group backward secrecy).*

Proof sketch. (a) K_i depends on the erased e_a, e_i; long-term keys only authenticate. (b) GK'_a is fresh and encrypted only under $\{K_i\}_{i \in S'}$; the removed member knows none of them. This is the property the base scheme loses by Lemma 1. (c) The joiner receives only the fresh GK'_a. □

12.5 Privacy

Theorem 12.6 (Anonymity, epoch-unlinkability, traceability)**.** *(i) An eavesdropper, or any entity other than the holder of* t*, learns nothing about* $RID_{\mathcal{E}}$ *from* $PID_{\mathcal{E}}$*. (ii) Requests from different epochs cannot be linked. (iii) The holder of* t *can recover* $RID_{\mathcal{E}}$ *from any* $PID_{\mathcal{E}}$*.*

Proof sketch. (i) $PID^2 = RID \oplus H_0(\rho T_{\text{pub}} \,\|\, ep)$ is hashed-ElGamal encryption; computing ρT_{pub} from $PID^1 = \rho P$ and $T_{\text{pub}} = tP$ is CDH. (ii) All values that appear in a request $(PID^1, PID^2, X, Y, E, R, v)$ are freshly random per epoch, because RG uses independent $\rho, y, x^{(ep)}$. (iii) TR1. □

Honest limitation

Within *one* epoch an entity uses one pseudonym, so its sessions in that epoch are linkable. This is a deliberate trade-off: it makes local eviction effective (an evicted pseudonym cannot be replaced until the next epoch). The epoch length sets the balance; the base paper's per-session pseudonyms resolvable only by the TA cannot support local eviction at all.

12.6 Replay and handover security

Theorem 12.7 (Replay resistance)**.** *A recorded request, ACK, bundle or handover message is rejected outside its session.*

Proof sketch. Requests sign (ID_a, E_a, ts_i), and E_a changes every beacon period. ACK and rekey entries carry ν and ts in their associated data. Bundles carry ts under τ. Tickets carry a single-use nonce $N_{\mathcal{M}}$ cached until *exp*, and $\tau_{\mathcal{M}}$ binds the fresh $E'_{\mathcal{M}}$ and ts. □

Theorem 12.8 (Handover)**.** *(i) Only $\mathcal{T}_a, \mathcal{T}_b$ can open or create a valid ticket. (ii) Only the member that holds $K_{\mathcal{M}}$ can use its ticket. (iii) $K'_{\mathcal{M}}$ is secret even against a later compromise of K_{ab}, $K_{\mathcal{M}}$ or long-term keys. (iv) $\mathcal{T}_b$ never learns $K_{\mathcal{M}}$.*

Proof sketch. (i) AEAD under K_{ab} with $ID_a \,\|\, ID_b$ as associated data. (ii) $\tau_{\mathcal{M}}$ requires $hk_{\mathcal{M}} = \mathsf{KDF}(K_{\mathcal{M}} \,\|\, \dots)$. (iii) $K'_{\mathcal{M}}$ mixes $e'_b e'_{\mathcal{M}} P$ (CDH, ephemeral). (iv) The ticket contains $hk_{\mathcal{M}}$, a one-way derivative of $K_{\mathcal{M}}$. □

12.7 Security comparison

Table 12.1: Security properties. ✓ holds, ✗ fails, ● partially. "Base (claimed)" is Table II of [1]; "Base (actual)" is our analysis.

	Property	Base (claimed)	Base (actual)	**Ours**
F1	Unforgeability (with valid proof)	✓	● proof empty (F. 5)	✓ Thm 12.1
F2	Conditional privacy	✓	✓	✓ Thm 12.6
F3	Session/group key establishment	✓	● leaks to members (F. 4)	✓ Thm 12.4
F4	Key-escrow resilience	✓	✗ (F. 2)	✓ Thm 12.2
F5	Scalability	✓	✗ $O(n^3)$ bytes (F. 4)	✓ $68n$ B
F6	Dynamic identity updating	✓	✓	✓ per epoch
F7	Forward secrecy	✓	✗ (F. 3)	✓ Thm 12.5
F8	Batch validation (sound)	✓	● no weights, no fallback	✓ Thm 12.3, Prop. 1
F9	Unlinkability	✓	✓ per session	● per epoch
F10	Non-repudiation	✓	✗ (F. 2)	✓ Thm 12.2
F11	Replay resistance	✓	✓	✓ Thm 12.7
N1	Works without online TA	–	✗ (F. 1)	✓ Prop. 2
N2	Secure revocation (leavers locked out)	–	✗ (F. 4)	✓ Thm 12.5
N3	Bounded cost under batch poisoning	–	✗	✓ Prop. 1
N4	Vehicle tier	–	✗ out of scope	✓ C2
N5	Multi-TUAV handover	–	✗	✓ Thm 12.8

CHAPTER 13

Formal Verification with ProVerif

The proof sketches of Chapter 12 argue each property separately. A protocol can still fail through an interaction between steps that no single proof considers. We therefore modelled the framework, and the base scheme's credential, in ProVerif 2.05 [23, 24]. ProVerif explores all interleavings of an unbounded number of sessions against a Dolev–Yao attacker. All models are in `final/proverif/`, and the full output of every run is kept next to them (`*.out`).

13.1 Modelling choices

- **Certificateless signatures.** The partial key is a function $\mathsf{partial}(s, (id, X))$ of the master key and of the identity *and* the entity's public value, mirroring $h = H_1(PID, ep, X, Y)$. A signature $\mathsf{clsign}(m, x, d)$ needs both the secret value x and the partial key d. Verification $\mathsf{clcheck}$ needs only id, $X = \mathsf{pubx}(x)$ and $P_{\text{pub}} = \mathsf{pubm}(s)$.
- **Diffie–Hellman** is modelled with the standard equation $\mathsf{exp}(\mathsf{exp}(g, a), b) = \mathsf{exp}(\mathsf{exp}(g, b), a)$; KDF, AEAD and MAC are modelled as perfect.
- **Type-I vs Type-II.** In the Type-I model the master key s is private. In the Type-II model s is public, so the attacker *is* the TA.
- **Forward secrecy** uses a second *phase* in which every long-term secret (and, for C1, even s) is published. Secrets sent in phase 0 must stay secret.
- **Single-use tickets** (C3) are modelled with a ProVerif table of used nonces.
- **The base scheme's credential** is abstracted as $\Gamma = \mathsf{gam}(s_i, \xi_i, R_{tu}, Q_{tu})$, where every argument except s_i is public, exactly as in equation (4.1).

13.2 Results

Table 13.1: ProVerif results. "Expected" is what the design predicts; every result matches it.

#	Model	Property	Expected	Result
Q1	C1, Type I	TUAV accepts UAV's registered key ⇒ UAV sent it (injective)	true	✓ true
Q2	C1, Type I	TUAV accepts the UAV's identity with *any* key ⇒ UAV sent it	true	✓ true
Q3	C1, Type I	UAV obtains a key ⇒ TUAV sent it to that UAV (injective)	true	✓ true
Q4	C1, Type I	secrecy of the group key GK_a	true	✓ true
Q5	C1, Type I	data under GK_a stays secret after *all* long-term keys leak	true	✓ true
Q6	C1, Type II	malicious TA: accept under the *registered* key ⇒ UAV sent it	true	✓ true
Q7	C1, Type II	malicious TA: accept the identity with *any* key ⇒ UAV sent it	false	● false
Q8	C1, Type II	data under the pairwise key K_i stays secret from the TA	true	✓ true
Q9	C1, Type II	group key secret from the TA	false	● false
Q10	Rekey	revoked, compromised member knows the *old* key (sanity)	known	✓ known
Q11	Rekey	revoked member cannot learn the *new* key	true	✓ true
Q12	C3	new TUAV accepts $\mathcal{M}$ ⇒ $\mathcal{M}$ started that handover (injective)	true	● non-inj. only
Q13	C3	$\mathcal{M}$ completes ⇒ new TUAV sent that response (injective)	true	✓ true
Q14	C3	secrecy of the new cell's group key	true	✓ true
Q15	C3	data in the new cell stays secret after K_{ab} and $K_{\mathcal{M}}$ leak	true	✓ true
Q16	Base scheme	TUAV/TA accept UAV ⇒ UAV sent it (TA's database leaked)	false	✗ **attack**

13.3 Interpretation

The base scheme (Q16). ProVerif finds an attack in which the attacker, knowing s_i from the TA's database, produces a credential the verifier accepts for a UAV that never sent anything. This is Finding 2, found automatically.

C1 against outsiders (Q1–Q5). All properties hold, including mutual authentication in both directions and forward secrecy. Q5 holds even when the master key s is published in phase 1, because session data depends on erased ephemeral exponents.

C1 against a malicious TA (Q6–Q9): the exact boundary. This is the most informative result.

- **Q6 holds**: the TA cannot sign under a UAV's *registered* key, because it lacks x. This is the escrow-freeness the base scheme lacks.
- **Q8 holds**: the TA cannot read data protected by the UAV's pairwise key K_i.
- **Q7 fails** (as predicted): the TA can create a *second* key X' for the same pseudonym, issue $\mathsf{partial}(s, (id, X'))$, and be accepted. This is key replacement, which every certificateless scheme permits (Girault level 2).
- **Q9 fails** (as predicted): the TA can enrol a phantom member and so receive the group key.

These four results replace a vague claim ("the TA cannot impersonate") with a precise one: *the TA cannot impersonate a registered key or read pairwise traffic; it can create detectable new keys.* Section 12 and the limitations in Chapter 17 are worded accordingly.

Revocation (Q10–Q11). After a member is removed and fully compromised, the new key stays secret. The base scheme fails this in the computational setting (Finding 4, reproduced in code). The polynomial-root attack is algebraic and cannot be expressed in ProVerif's symbolic model, which is why we demonstrate it with code rather than ProVerif.

Handover (Q12–Q15). Secrecy and forward secrecy hold, and so does the member-side injective agreement (Q13). For Q12, ProVerif proves non-injective agreement but cannot prove injectivity. The reason is that in ProVerif a table lookup and the later insert are not atomic, so two parallel copies of the new TUAV can both accept one replayed ticket before either records the nonce. In our implementation the TUAV processes handover requests sequentially, so check-and-insert is atomic and the replay is rejected (test `test_c3_handover`). We report the ProVerif result as it is. Even in the abstract model, a replayed ticket gives the attacker nothing: by Q13–Q15 it cannot complete the session or learn any key.

Honest limitation

ProVerif works in the symbolic model: primitives are perfect and the attacker cannot exploit algebra beyond the stated equations. Its results complement, and do not replace, the computational arguments of Chapter 12. The models abstract timestamps, epochs and the batch equation; the batch test's soundness rests on Theorem 12.3 and is checked in code (experiment E7).

CHAPTER 14

Implementation

The base paper evaluates only by counting operations. We implemented both schemes so that every finding and every claim of our framework can be *executed*, and so that both schemes can be timed on the same machine. The code is in `final/` (about 1 800 lines including tests, experiments and ProVerif models).

14.1 Design principles

P1. Same curve, same library for both schemes. Both run on BN254 through `py_ecc` [25] (pure Python). The group $\mathbb{G}$ of our scheme is BN254's $\mathbb{G}_1$, which is exactly the group the base scheme pairs. Any measured difference therefore comes from the protocols, not from the tooling.

P2. Step labels as in the papers. Every function is named after the step it implements (IS, US, GA, GD, DU for the base scheme; RG, UA, RK, VA, HO, TR for ours), so the code can be read next to Chapters 3 and 7–11.

P3. Every operation is counted. The wrapper `cutcord/ec.py` counts scalar multiplications, small multiplications, additions, pairings and $\mathbb{G}_T$ exponentiations, so measured times can be cross-checked against operation counts (Chapter 16).

P4. Standard symmetric primitives from the `cryptography` library [26]: HKDF-SHA256, AES-128-GCM, HMAC-SHA256.

14.2 Structure

Table 14.1: Code layout.

File	Lines	Content
`cutcord/ec.py`	105	group operations with counters, domain-separated hashes $H(\cdot)$ and $H_q(\cdot)$, size table
`cutcord/sym.py`	35	KDF, AEAD seal/open, MAC
`cutcord/ours.py`	451	TA (ST, RG, TR), credentials and (batch) signatures, `Member` (UAV/vehicle), `TUAV` (UA, RK, VA, HO)
`cutcord/base.py`	172	base scheme IS–DU, CRT polynomial, satellite `Link`, attacks of Findings 2 and 4
`tests/`	236	20 tests (Table 14.2)
`experiments/run_all.py`	392	experiments E0–E7 → `results/*.csv`, `summary.json`
`experiments/make_figures.py`	149	figures for this report and the paper
`proverif/`	266	five models and a shared library (Chapter 13)

14.3 Realising the base scheme

- **Symmetric pairing on an asymmetric curve.** The base paper writes $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_1 \to \mathbb{G}_2$; BN254 offers $e : \mathbb{G}_2 \times \mathbb{G}_1 \to \mathbb{G}_T$. Every pairing in Step GA2 has, as one argument, a value the TA can compute in $\mathbb{G}_2$ from scalars it knows (P, $s_i P$, $s_{tu} P$). We place that argument in $\mathbb{G}_2$ and keep the protocol otherwise verbatim. The test `test_ga3_accepts_honest_batch` confirms that GA3 balances.

- **Declared reading of σ_i in GD1–GD6.** As in Finding 4(a), the CRT step needs integer moduli. We use $\sigma_i^{\text{int}} = \text{nextprime}(H(\sigma_i))$ of 160 bits, which makes the moduli pairwise coprime and lets every honest member recover the key (`test_crt_key_distribution`).
- **GA1 forwards** Γ_i, which GA2 needs although the paper's GA1 does not list it.

14.4 Realising our framework

- **Credentials** (`TA.register`) implement RG1–RG4 with one credential per epoch, a Schnorr proof of possession and the RG4 check.
- **Batch verification** (`batch_verify`) implements equation (8.1) with 64-bit weights; `TUAV.verify_window` adds the filter, the fallback and the adaptive mode switch (Figure 8.2).
- **Key delivery and rekeying** (`TUAV.admit`, `TUAV.rekey`) seal $GK_a \,\|\, \nu$ under each member's K_i, with the version and timestamp as associated data.
- **Members** keep a *pending* request (its ephemeral e) separately from the *established* session (K, epoch). The ephemeral is erased as soon as the key is derived. An early version stored both in one place, and our tests caught the resulting bug.
- **Vehicles** (`Member(role="veh")`, `Member.relay`, `TUAV.process_bundle`) and **handover** (`link_offer/acce` `issue_ticket`, `accept_handover` with a nonce cache) follow VA1–VA5 and HO0–HO4.

14.5 Tests

Table 14.2: The 20 tests (`python -m pytest`, all passing).

Test	What it establishes
Base scheme	
`ga3_accepts_honest_batch`	GA3 and per-UAV checks balance (implementation is faithful)
`crt_key_distribution`	every honest member recovers κ_{tu}
`finding2_ta_forges_any_uav`	a TA-built request passes GA3 (Finding 2)
`finding5_batch_cancellation`	$\Gamma_1 + \Delta, \Gamma_2 - \Delta$ pass the batch, fail individually (Finding 5)
`finding1_one_bad_request_rejects_all`	one bad request rejects the whole batch
`finding4_revoked_member_recovers_new_key`	the revocation attack recovers κ^* (Finding 4)
Our framework	
`registration_keys_consistent`	$sk \cdot P = pk$
`c1_join_and_group_key`	one batch check admits all, everyone gets the same GK_a
`ta_cannot_sign_without_x`	knowing d is not enough to sign (escrow-freeness)
`key_replacement_fails`	an outsider replacing X is rejected (Type I)
`batch_rejects_cancelling_errors`	random weights defeat cancellation
`fallback_keeps_honest_and_evicts_bad`	failed batch → honest admitted, bad evicted, mode switch
`replayed_request_rejected_in_new_beacon`	requests are bound to E_a
`rekey_locks_out_removed`	removed member cannot open any new entry
`c2_vehicle_admission_via_relay`	vehicles keyed through a relay; $VK_a \neq GK_a$
`c2_outsider_bundle_dropped`	bundles from non-members are dropped
`c3_handover`	ticket handover works; replayed ticket rejected
`c3_evicted_member_cannot_hand_over`	eviction lists propagate between TUAVs
`tracing_and_global_revocation`	TR1 recovers *RID*; TR2 bans future epochs only
`pseudonyms_unlinkable_across_epochs`	no value repeats across epochs

Honest limitation

`py_ecc` is pure Python and not constant-time: it suits a reference implementation and comparative measurement, not deployment. BN254 offers roughly 100-bit security today, and our scheme needs no pairing, so a deployment would use a 128-bit curve such as P-256 or Curve25519 with a constant-time library.

CHAPTER 15

Analytical Comparison with the Base Paper's Constants

Method. For a fair comparison we use *exactly* the operation timings of the base paper (MIRACL, 80-bit level, Section VI-A of [1]): $T_P = 7.2232$, $T_{SM\text{-}P} = 2.6784$, $T_{PA\text{-}P} = 0.0342$, $T_{SM\text{-}E} = 0.9018$, $T_{SSM\text{-}E} = 0.0475$, $T_{PA\text{-}E} = 0.0107$, $T_{ME} = 0.6256$, $T_{SH} = 0.0012$ ms. Symmetric operations (KDF, AEAD, MAC on short inputs) are charged as T_{SH}. Sizes also follow the base paper: $|\mathbb{G}| = 40$ B, $|\mathbb{G}_1| = |\mathbb{G}_2| = 128$ B, hash 20 B, timestamp 4 B; an AEAD ciphertext of a 16-byte key with version, nonce and tag is 48 B. All numbers are produced by `scripts/compute_results.py`.

How this chapter relates to Chapter

This chapter prices both schemes with the base paper's own MIRACL timings (80-bit level), the same method the base paper used. Chapter 16 then *measures* both schemes on one curve and one library. The two views agree in direction. They differ in magnitude because pure-Python pairings are relatively more expensive than MIRACL's (about 41× a scalar multiplication versus 8×).

15.1 Computation

Table 15.1: Computation cost (ms). "Base (honest)" adds the TA's GA2 work (29.7414 ms per UAV) and places Γ_i in $\mathbb{G}_1$.

Operation	Base (reported)	Base (honest)	**Ours**
UAV signing	1.8203	5.3970	**1.8048** $(2T_{SM\text{-}E} + T_{SH})$
Single verification	2.1578	31.8992 + RTT	**2.7399** $(3T_{SM\text{-}E} + 3T_{PA\text{-}E} + 2T_{SH})$
Group verification, n	$1.2524n + 0.9054$	$30.9938n + 0.9054$ + RTT	$\mathbf{0.9838n + 1.8036}$
at $n = 120$	151.19	3720.16 + RTT	**119.86**
Group verification + key delivery, $n = 120$	not reported	$\geq$ 3720.16 + RTT	**228.36** $(1.8880n + 1.8036)$
Member: receive key (incl. beacon check)	not reported	–	3.6441

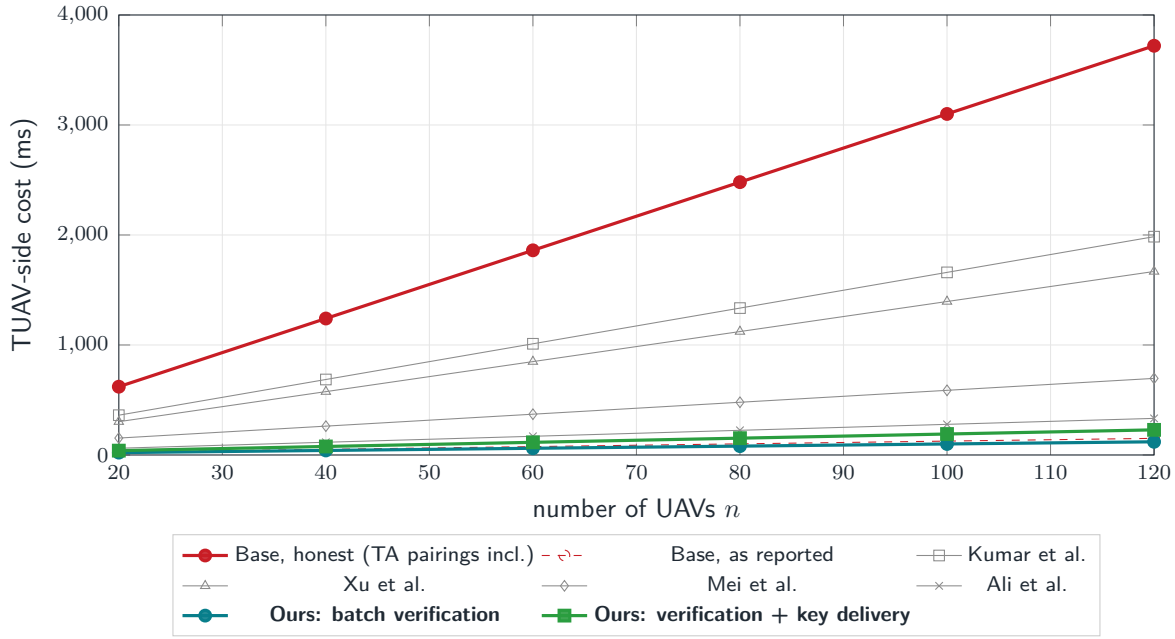


Figure 15.1: Group authentication cost versus n (satellite RTT *not* added to the base scheme here). Counted honestly, the base scheme is the most expensive of all six.

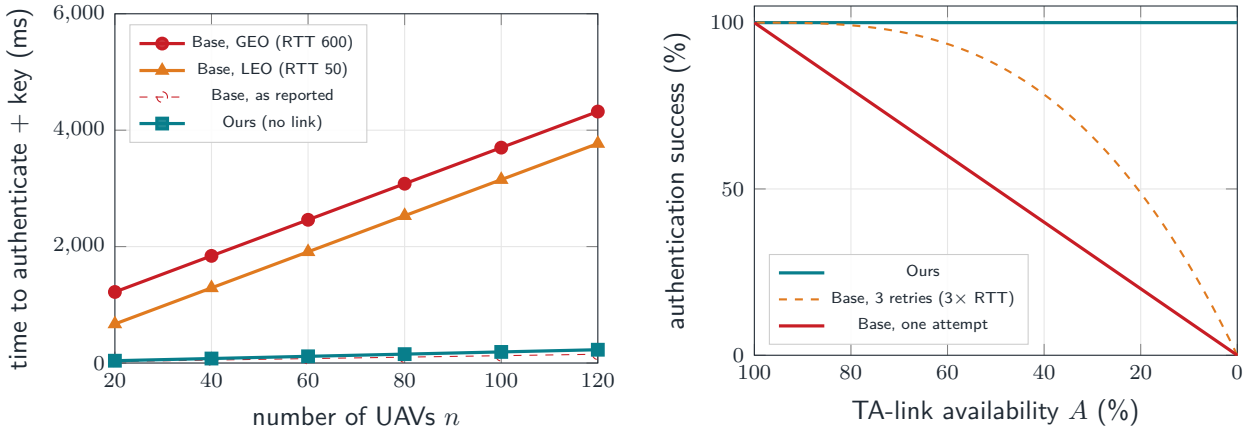


Figure 15.2: Left: end-to-end time with the satellite round trip charged to the base scheme. Right: *analytical* availability. The base scheme falls to zero with the link; ours does not depend on it (Proposition 2).

Why our scheme is faster even though the TUAV now verifies

The base scheme looks cheap only because the expensive part (three pairings per UAV, about 21.7 ms) runs on the TA and is left out of the count. Our verification needs no pairings at all: a batch costs about one elliptic-curve scalar multiplication (0.90 ms) per UAV. Removing the TA from the loop therefore *reduces* the total work while also removing the wide-area round trip. The extra cost we *do* add is the per-member ECDH for key delivery (0.90 ms per UAV), which buys forward secrecy and secure revocation.

15.2 Communication

Table 15.2: Communication overhead (bytes).

Message	Base (reported)	Base (honest)	**Ours**
Join request (per UAV)	104	192 ($\Gamma_i \in \mathbb{G}_1$)	248
Satellite traffic (per UAV)	–	192 up + 512 down	**0**
Key delivery, $n = 120$ (all members)	–	16.72 MiB	**8.0 KiB** ($68n + 4$)
TUAV beacon (per period)	–	172 ($Q_{tu} \in \mathbb{G}_1$)	204
Rekey after one leave, $n = 120$	–	16.3 MiB	7.9 KiB (flat) / **680 B** (LKH)

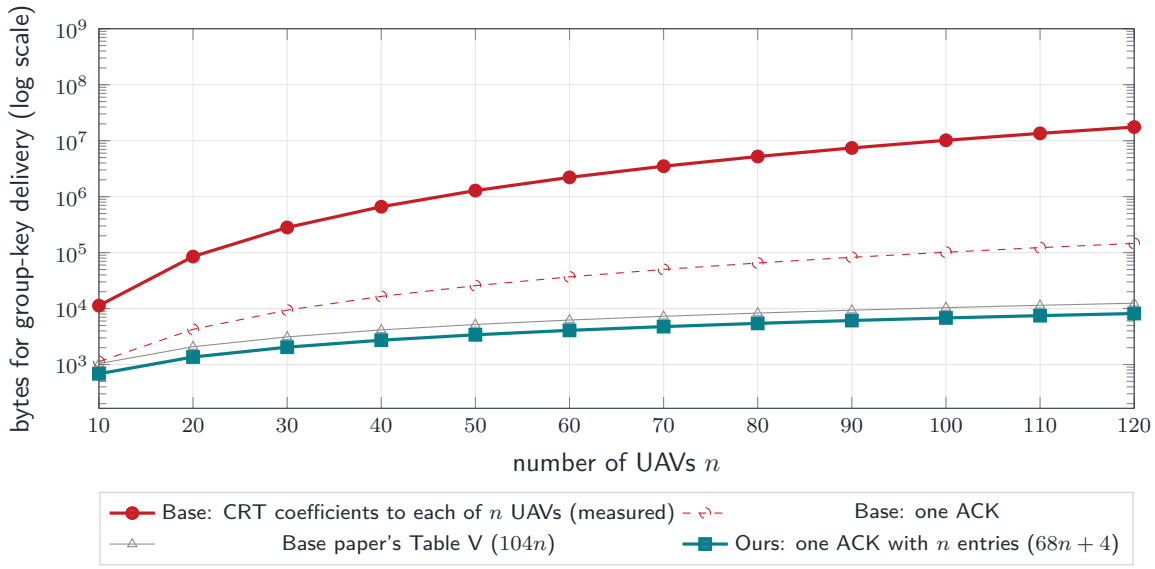


Figure 15.3: Group-key delivery traffic. The base scheme grows as $O(n^3)$ bytes; ours grows linearly, and at $n = 120$ is about 2150× smaller.

The trade-off we accept

Our join request is larger (248 B against 192 B honest or 104 B reported) because it carries the public-key material X_i, Y_i (so that no directory or TA is needed to verify it) and the ephemeral E_i (forward secrecy). We think this is the right trade: a few hundred extra bytes per UAV to remove the satellite traffic, the TA dependence and a key-delivery broadcast that is megabytes in size.

15.3 Handover

Table 15.3: Admission in a new cell: full C1/C2 admission versus a C3 ticket.

	Full re-admission	**C3 handover**	saving
New TUAV computation	3.6441 ms	**1.8084 ms**	50%
Member computation	5.4489 ms	**1.8072 ms**	67%
Signature verifications at the new TUAV	1	**0**	–
Bytes on air (member ↔ new TUAV)	248 + 68 + beacon	301 + 92	comparable
TA involvement	none	none	–

The bigger benefit is *latency and continuity*. The member does not need to wait for the new TUAV's beacon period, generate a signature or pass a verification window, and the eviction state follows it across cells.

CHAPTER 16

Measured Evaluation

This chapter reports experiments E0–E7 run on our implementation (Chapter 14). Every number in the text comes from `final/results/` through generated macros, so the text cannot drift from the data.

16.1 Setup and methodology

- **Platform.** Python 3.14, `py_ecc` 8.0.0 (pure Python, BN254), `cryptography` 46.0.4, Windows 11 laptop.
- **Fairness.** Both schemes use the same curve, library and process. For each swarm size n the two schemes run back to back, so the ratio is measured under the same machine state.
- **Robustness.** The laptop's speed drifted by up to a factor of five during the day, sometimes within a single run (power management). We therefore report two ratios side by side. (i) The *measured* ratio of the two schemes, run back to back for each n. (ii) A *load-independent* ratio: the exact operation counts recorded by the code, priced with the same micro-benchmark costs, so that machine speed cancels out. Repeated measurements (E3, E5, E6) are medians of three runs.
- **Network.** Satellite round-trip time and link availability are parameters (Finding 1); they are added, never measured.

16.2 E0: primitive costs

Table 16.1: Median cost of primitives on the measurement machine (E0).

Operation	Median (ms)	Runs
G1 scalar multiplication	6.8657	60
G1 small (64-bit) multiplication	1.3922	60
G1 point addition	0.0199	60
G2 scalar multiplication	31.0114	10
bilinear pairing	1403.2725	10
GT exponentiation	88.5351	10
SHA-256 hash	0.0043	60
HKDF-SHA256	0.0318	60
AES-128-GCM seal (20 B)	0.017	60

Absolute values in Table 16.1 reflect the machine state of the last run. Measured *interleaved*, so that both share one machine state, a pairing costs about 44 scalar multiplications here, against about 8 in the MIRACL figures used by the base paper. Pure-Python measurements therefore *favour* our pairing-free design more than optimised C would; Chapter 15 gives the MIRACL-based view, which points the same way.

16.3 E1: availability under TA-link loss

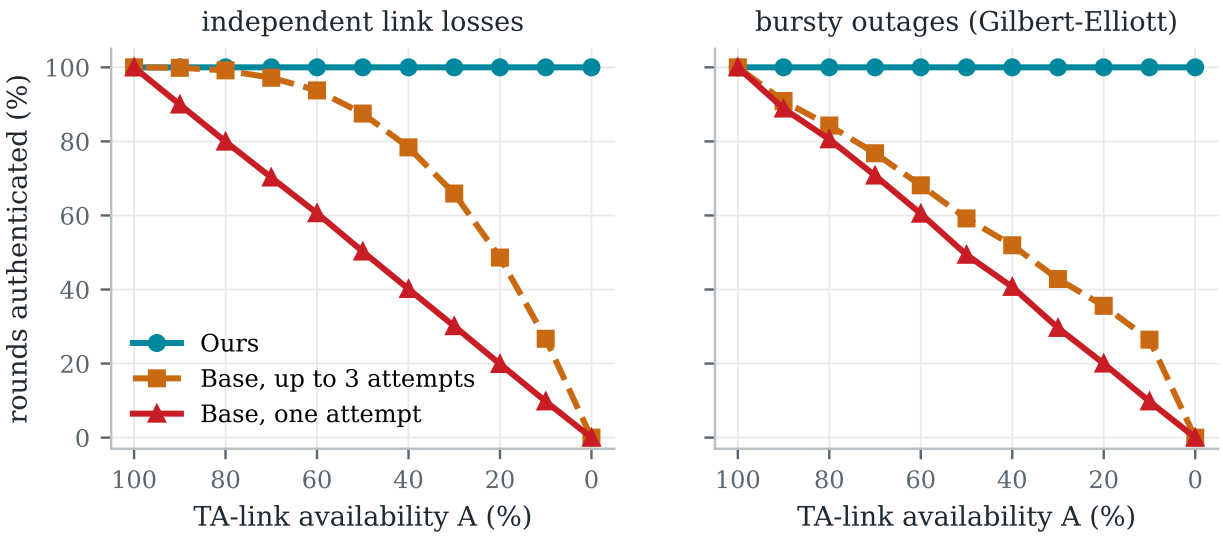


Figure 16.1: Share of authentication rounds that succeed as TA-link availability falls (20 000 rounds per point), for independent losses and for bursty outages (Gilbert–Elliott, mean outage 10 rounds).

The base scheme succeeds in a round only if the satellite link is up. With $A = 50\%$ it succeeds in 50.3% of rounds, and 87.5% if it may retry three times, at three round-trip times. Under bursty outages, which are typical of congestion or jamming, retries help much less (59.2%) because consecutive attempts fail together. At $A = 0$ the base scheme admits nobody. Ours does not use the link, so it stays at 100% by construction (Proposition 2). This experiment illustrates the dependency rather than discovering it.

16.4 E2: time to authenticate and key a swarm

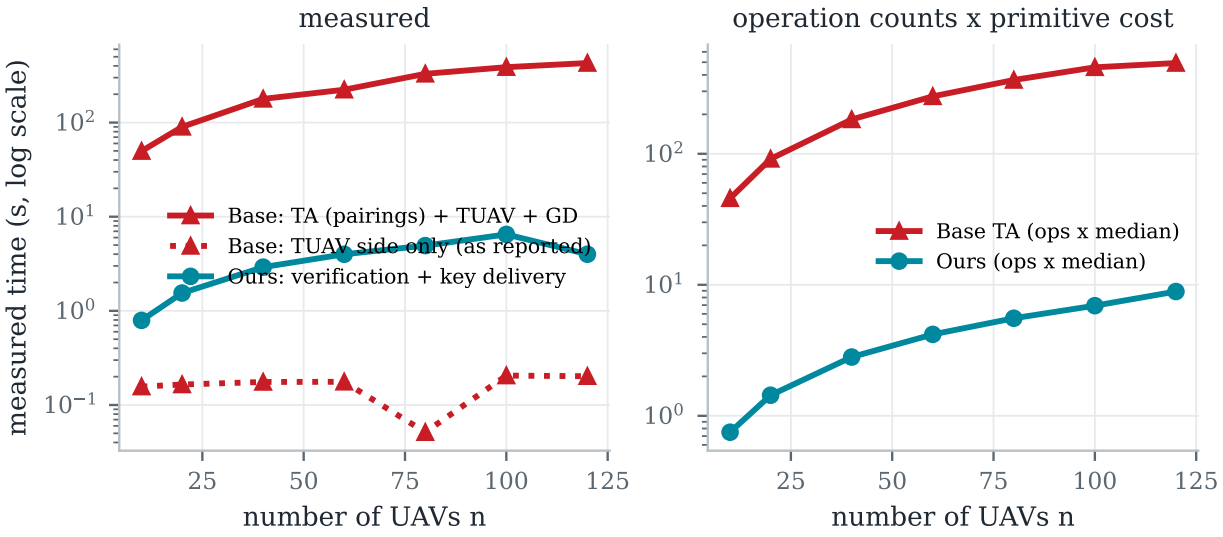


Figure 16.2: Left: measured time for the TUAV to authenticate and key n UAVs (ours) against the base scheme's GA1–GD4 with the TA's work included, and the TUAV-only part the base paper reports. Right: load-independent estimate from operation counts. Log scale; satellite RTT not added. The measured $n = 120$ point ran across a change in machine speed (see text).

Table 16.2: E2 measurements (ms). Our time includes verification *and* key delivery. The base scheme's time is split into TA (GA2), TUAV (GA3) and key distribution (GD).

n	Ours verify	Ours keys	**Ours total**	Base TA	Base TUAV	Base GD	**Base total**	Pairings
10	474	319	792	49,236	157	357	49,750	30
20	956	591	1,546	89,263	165	590	90,018	60
40	1,683	1,229	2,912	176,309	176	1,941	178,425	120
60	2,324	1,668	3,992	221,276	176	1,779	223,232	180
80	2,822	2,108	4,931	328,532	51	788	329,371	240
100	3,564	2,926	6,490	384,340	205	3,047	387,592	300
120	2,969	1,025	3,993	424,971	202	3,553	428,726	360

For $n = 100$ our TUAV authenticates and keys the swarm in 6.49 s (64.9 ms per UAV). The base scheme needs 388 s, of which the TA's pairings take 384 s (3.84 s per UAV), before any satellite delay. Across all swarm sizes the base scheme does **55–66× more work** by operation count (median 65×). The measured ratios for $n \leq 100$ lie in the same band (56–67×). The $n = 120$ point was re-measured while the machine changed speed between the two schemes. Its measured ratio (107×) is therefore not used; its operation-count ratio is in the band. The TUAV-only part that the base paper reports is just 0.05% of the base scheme's real work. This confirms Finding 6 by measurement: the offload does not remove the cost, it relocates it to the TA, behind the fragile link.

16.5 E3: batch poisoning

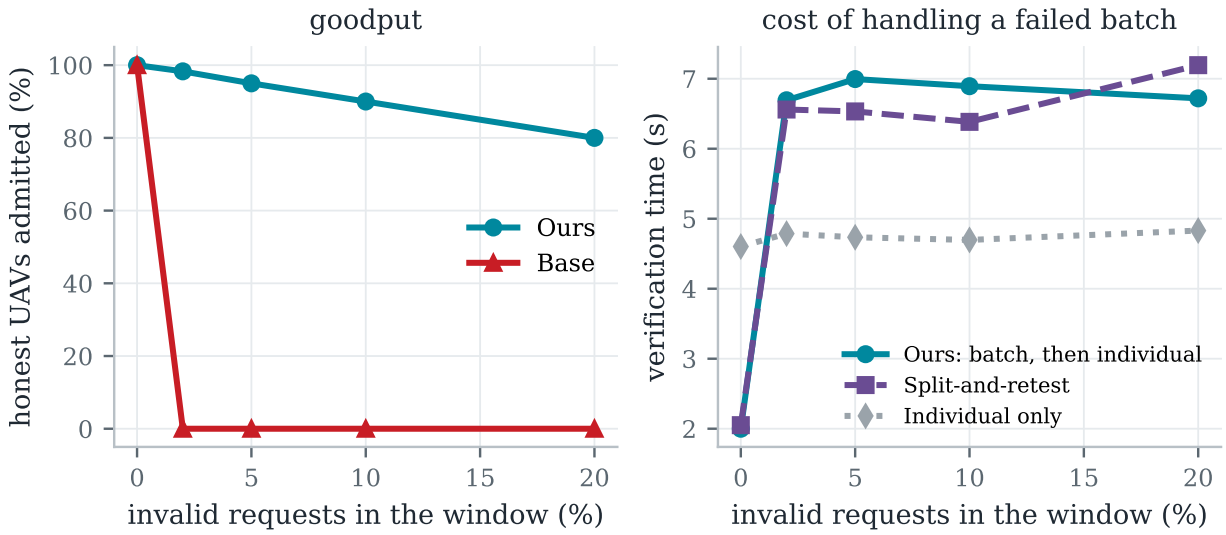


Figure 16.3: Left: share of honest UAVs admitted as the share of invalid requests grows ($n = 60$). Right: verification time of three strategies for handling a failed batch.

In the base scheme a single invalid request makes the whole batch fail, and nobody is admitted (verified for $n = 10$ and in E7). With our rule every honest UAV is admitted at every attack rate (80% goodput when 20% of requests are invalid, i.e. all honest ones). The cost is bounded as Proposition 1 predicts: a clean window takes 2.00 s, and the worst window (batch fails, then each request is checked) 7.00 s. After one failure the TUAV switches to individual mode, whose cost is independent of the attack rate. Split-and-retest offers no advantage here, which confirms the analytical finding of Section 8.2: with a pairing-free scheme, a batch is only a few times cheaper per signature, so re-testing halves does not pay.

16.6 E4: communication

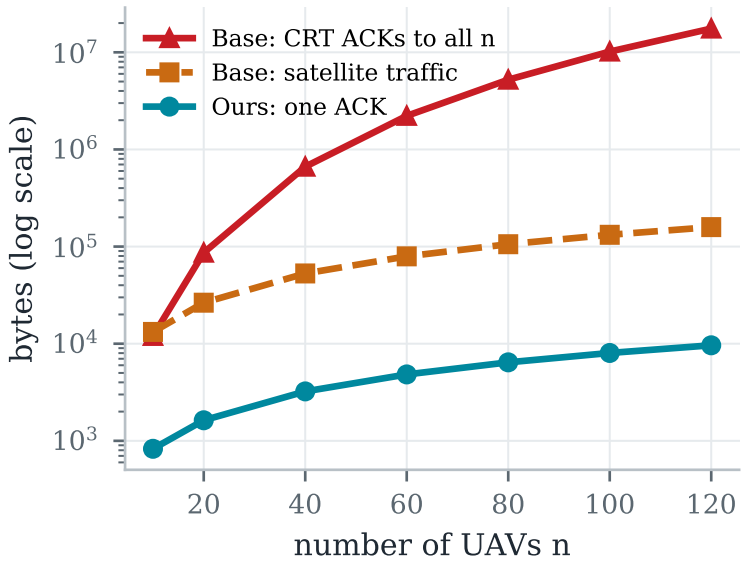


Figure 16.4: Group-key traffic and the base scheme's satellite traffic, measured from the encodings of our implementation (compressed points, 32-byte hashes).

For $n = 120$ the base scheme's acknowledgements carry 17.5 MB of CRT coefficients in total. Ours sends one acknowledgement of 9.6 KB, 1,823× less. The base scheme also moves 158.6 KB over the satellite link for one authentication round, against zero for ours. Our join request is larger (225 B against 137 B) because it carries the public-key material and an ephemeral value; Chapter 17 lists this as a limitation.

16.7 E5: the vehicle tier

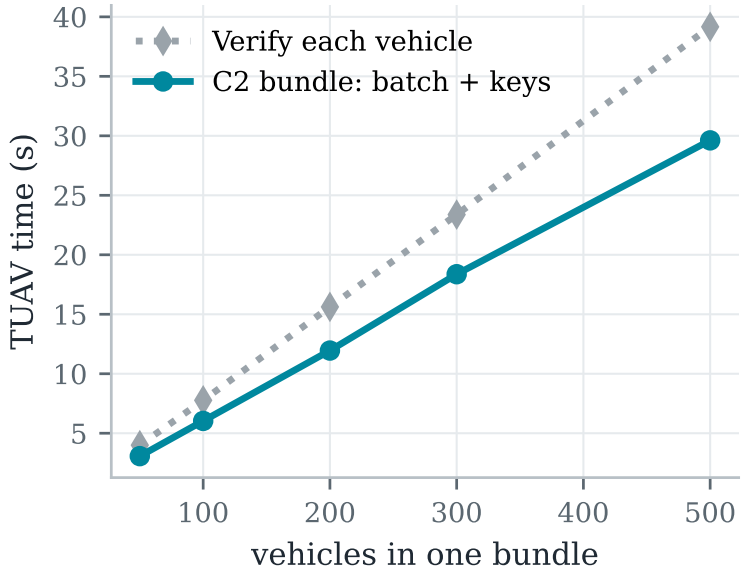


Figure 16.5: TUAV time to admit a bundle of vehicles relayed by one UAV (batch verification *and* key delivery) against verifying the same requests one by one.

A bundle of 500 vehicles is admitted, *including* key derivation and encryption, in 29.6 s. Verifying the same requests one by one, without keys, takes 39.2 s. A bundle from a non-member relay is dropped after one MAC check in at most 15 μs, regardless of its size, so floods of forged vehicle requests never reach signature verification.

16.8 E6: handover

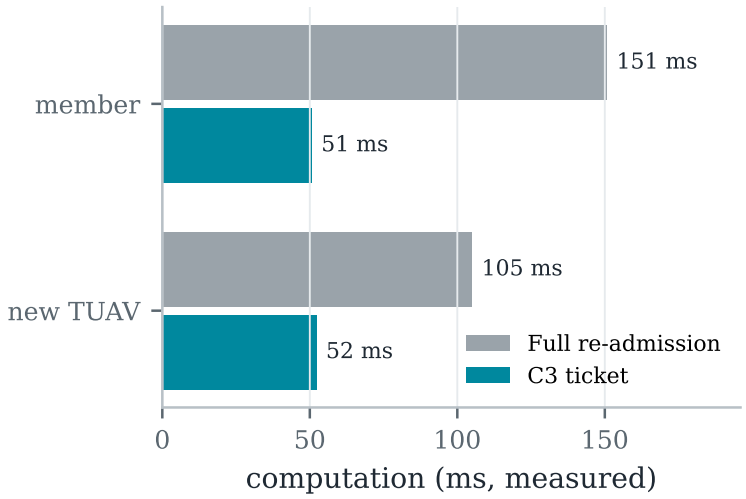


Figure 16.6: Computation for one handover: full re-admission in the new cell against a C3 ticket (median of 25 runs).

A ticket handover costs the new TUAV 52.3 ms instead of 105.0 ms, and the member 50.7 ms instead of 150.8 ms. In a one-hour mobility run (3 cells of 2 km, 30 UAVs at 10–20 m/s) there were 557 cell crossings. With a 1 s beacon period the mean service gap per crossing falls from 756 ms (wait for the next beacon, then full admission) to 103 ms. The TUAVs' total admission CPU falls from 58.5 s to 29.1 s.

16.9 E7: attacks, executed

Table 16.3: Attack outcomes, executed against both implementations.

Attack	Base scheme	Ours
TA forges a request for a registered UAV	✗ succeeds	✓ fails
Two cancelling errors pass the batch check	✗ succeeds	✓ fails
Revoked UAV recovers the new group key	✗ succeeds	✓ fails
One invalid request blocks all honest UAVs	✗ yes	✓ no
Old request replayed under a new beacon	–	✓ rejected
Replayed handover ticket	–	✓ rejected (test)
Bundle from a non-member relay	–	✓ dropped (E5)

16.10 Summary

Every measurement points the same way as the analysis. The base scheme's apparent efficiency comes from not counting the TA. Its key traffic grows as n^3. Its batch has no failure path. Ours removes the TA from the field, is faster in total, is linear in traffic, degrades gracefully under poisoning, and extends to vehicles and handover at modest cost.

CHAPTER 17

Discussion, Limitations and Conclusion

17.1 What the evidence supports

Our claim, stated precisely

The published scheme does not achieve infrastructure independence, escrow resilience, non-repudiation, forward secrecy or secure revocation, and its reported costs omit the dominant work. These properties can be recovered *together*, using established primitives, with lower total computation and linear communication. The same offline trust root then extends to vehicles and to multi-TUAV handover. We do not claim new cryptographic primitives.

Each part of this claim rests on two independent kinds of evidence:

Claim	First evidence	Second evidence
Base scheme needs the TA online	protocol inspection	availability model (E1)
TA impersonates base-scheme UAVs	code (E7)	ProVerif Q16
Revoked UAV keeps the key (base)	code (E7, test)	algebraic proof (Lemma 1)
Our TA cannot sign under registered keys	code (test)	ProVerif Q6, Theorem 12.2
Our revocation locks leavers out	code (E7, test)	ProVerif Q11
Ours is cheaper overall	measured (E2)	analytical (Chapter 15)
Bounded behaviour under poisoning	measured (E3)	Proposition 1

17.2 Limitations

L1. Key replacement by a malicious TA (Girault level 2). The TA cannot sign under a registered key, but it can issue a second key for a pseudonym or enrol a phantom member (ProVerif Q7, Q9). This is inherent to certificateless cryptography. It is detectable, and a manufacturer-certified device key would remove it.

L2. Captured TUAV. With the TA offline, the TUAV is the most trusted field entity. A captured TUAV learns its cell's group and pairwise keys and its link keys until it is evicted. It cannot forge members or learn their secret values. Threshold admission across TUAVs is future work.

L3. Linkability within an epoch is the price of effective local eviction; the epoch length tunes it.

L4. Pseudonym pool. Each epoch credential takes about 184 bytes; 288 epochs (three days of 15-minute epochs) take about 53 KB and must be loaded before deployment.

L5. Deferred global revocation needs the TA to be reachable at some point; local eviction covers the gap until the epoch ends.

L6. Larger join request than the paper reports (225 B versus 137 B under our encoding; 248 B versus 104 B under the paper's), because ours carries X, Y and an ephemeral E.

L7. Measurement platform. Timings come from a pure-Python library on a laptop whose speed varied during the runs. We therefore report ratios measured back to back and cross-check them with operation counts (Chapter 16). The absolute values are not those of an optimised C implementation.

L8. Models. Network behaviour (link losses, mobility) is simulated with simple stochastic models rather than a radio simulator. The ProVerif results are symbolic, and the reductions are proof sketches.

L9. Vehicle routine beacons protected by the shared VK_a can be forged by any vehicle holding it; events that matter are signed.

17.3 Conclusion

The base paper addresses a real problem, secure vehicular networking after a disaster has destroyed the roadside infrastructure, and its TUAV architecture is a good answer. Its protocol, however, places the cloud TA inside every operation. We showed, through analysis, code and formal verification, that this makes authentication stop when the satellite link fails, lets the TA impersonate every UAV, lets a revoked UAV keep receiving group keys, and hides the dominant cost.

Cutting the Cord keeps the architecture and moves the trust root offline. Escrow-free certificateless credentials let a TUAV verify UAVs and vehicles itself, in a sound batch with a bounded fallback. Per-member ECDH keys make group-key delivery linear, forward-secure and revocation-safe. Encrypted single-use tickets hand members between TUAVs without the TA. ProVerif verifies the intended properties and pins down the one residual TA power. On the same curve and library, our TUAV authenticates and keys a swarm one to two orders of magnitude faster than the base scheme's TA-inclusive cost, with no satellite traffic and kilobytes instead of megabytes of key traffic.

17.4 Future work

- Threshold trust across TUAVs, so that one captured TUAV cannot admit members alone.
- Level-3 trust: bind each RG1 record to a manufacturer-certified device key.
- A constant-time implementation on a 128-bit curve, measured on embedded UAV hardware.
- Integration with a radio and mobility simulator (for example ns-3) and a field trial with a tethered drone.
- Post-quantum credentials (lattice-based signatures) for long-lived deployments.

Bibliography

[1] H. Tan, W. Zheng and P. Vijayakumar, "Secure and efficient authenticated key management scheme for UAV-assisted infrastructure-less IoVs," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 6, pp. 6389–6400, Jun. 2023, doi: 10.1109/TITS.2023.3252082.

[2] Q. Mei, H. Xiong, J. Chen, M. Yang, S. Kumari and M. K. Khan, "Efficient certificateless aggregate signature with conditional privacy preservation in IoV," *IEEE Syst. J.*, vol. 15, no. 1, pp. 245–256, 2021.

[3] P. Kumar, S. Kumari, V. Sharma, X. Li, A. K. Sangaiah and S. H. Islam, "Secure CLS and CL-AS schemes designed for VANETs," *J. Supercomput.*, vol. 75, no. 6, pp. 3076–3098, 2019.

[4] I. Ali, M. Gervais, E. Ahene and F. Li, "A blockchain-based certificateless public key signature scheme for vehicle-to-infrastructure communication in VANETs," *J. Syst. Archit.*, vol. 99, Art. no. 101636, 2019.

[5] Z. Xu, D. He, N. Kumar and K.-K. R. Choo, "Efficient certificateless aggregate signature scheme for performing secure routing in VANETs," *Secur. Commun. Netw.*, vol. 2020, Art. no. 5276813, 2020.

[6] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. ASIACRYPT*, LNCS 2894, 2003, pp. 452–473.

[7] M. Girault, "Self-certified public keys," in *Proc. EUROCRYPT*, LNCS 547, 1991, pp. 490–497.

[8] Y. Han, W. Song, Z. Zhou, H. Wang and B. Yuan, "ECLAS: An efficient pairing-free certificateless aggregate signature for secure VANET communication," *IEEE Syst. J.*, vol. 16, no. 1, pp. 1637–1648, 2022.

[9] C. P. Schnorr, "Efficient signature generation by smart cards," *J. Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.

[10] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.

[11] M. Bellare, J. A. Garay and T. Rabin, "Fast batch verification for modular exponentiation and digital signatures," in *Proc. EUROCRYPT*, LNCS 1403, 1998, pp. 236–250.

[12] J. Pastuszak, D. Michałek, J. Pieprzyk and J. Seberry, "Identification of bad signatures in batches," in *Proc. PKC*, LNCS 1751, 2000, pp. 28–45.

[13] L. Law and B. J. Matt, "Finding invalid signatures in pairing-based batches," in *Cryptography and Coding*, LNCS 4887, 2007, pp. 34–53.

[14] G. M. Zaverucha and D. R. Stinson, "Group testing and batch verification," in *Proc. ICITS*, LNCS 5973, 2009, pp. 140–157.

[15] C. K. Wong, M. Gouda and S. S. Lam, "Secure group communications using key graphs," *IEEE/ACM Trans. Netw.*, vol. 8, no. 1, pp. 16–30, 2000.

[16] D. Wallner, E. Harder and R. Agee, "Key management for multicast: Issues and architectures," RFC 2627, 1999.

[17] G.-H. Chiou and W.-T. Chen, "Secure broadcasting using the secure lock," *IEEE Trans. Softw. Eng.*, vol. 15, no. 8, pp. 929–934, 1989.

[18] M. A. Khan *et al.*, "An efficient and provably secure certificateless key-encapsulated signcryption scheme for flying ad-hoc network," *IEEE Access*, vol. 8, pp. 36807–36828, 2020.

[19] Z. Su, Y. Wang, Q. Xu and N. Zhang, "LVBS: Lightweight vehicular blockchain for secure data sharing in disaster rescue," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 19–32, 2022.

[20] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.

[21] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007.

[22] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.

[23] B. Blanchet, "An efficient cryptographic protocol verifier based on Prolog rules," in *Proc. IEEE CSFW*, 2001, pp. 82–96.

[24] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Found. Trends Priv. Secur.*, vol. 1, no. 1–2, pp. 1–135, 2016.

[25] Ethereum Foundation, "py_ecc: Elliptic curve cryptography in Python," version 8.0.0. [Online]. Available: https://github.com/ethereum/py_ecc

[26] Python Cryptographic Authority, "cryptography," version 46.0.4. [Online]. Available: https://cryptography.io

CHAPTER A

Reproducing the Results

All commands run from the folder `final/`.

```
python -m pip install py_ecc cryptography sympy matplotlib pytest
python -m pytest -q                              # 20 tests
python experiments/run_all.py                    # E0-E7 -> results/*.csv, results/summary.json
python experiments/run_all.py --only=e2,e3       # re-run selected experiments
python experiments/make_figures.py               # figures/*.pdf, *.png
cd proverif && ../tools/proverif2.05/proverif.exe -lib common c1_typeI.pv
cd report  && pdflatex final_report && pdflatex final_report
```

The analytical numbers of Chapter 15 come from `phase2/scripts/compute_results.py`. ProVerif 2.05 for Windows is the official binary distribution (`proverifbin2.05.tar.gz`).

CHAPTER B

ProVerif Models

B.1 Shared library

```
(* Shared declarations for the Cutting-the-Cord models (ProVerif 2.05).
   Certificateless signature: signing needs the entity's secret value x AND the TA-issued
   partial key d = partial(s, id); verification needs only id, X = pubx(x) and Ppub = pubm(s). *)

free c: channel.

type G.
type exponent.
type key.
type master.
type secretv.
type partialk.

(* Diffie-Hellman *)
const g: G [data].
fun exp(G, exponent): G.
equation forall x: exponent, y: exponent; exp(exp(g, x), y) = exp(exp(g, y), x).

(* KDF, AEAD, MAC *)
fun kdf(G, bitstring): key.
fun kdf2(key, bitstring): key.
fun senc(bitstring, key): bitstring.
reduc forall m: bitstring, k: key; sdec(senc(m, k), k) = m.
fun mac(bitstring, key): bitstring.
fun k2b(key): bitstring [data, typeConverter].

(* certificateless credentials *)
fun pubm(master): G.
fun pubx(secretv): G.
fun partial(master, bitstring): partialk.          (* d bound to (id, X) via h = H1(PID, ep, X, Y) *)
fun clsign(bitstring, secretv, partialk): bitstring.
reduc forall m: bitstring, x: secretv, s: master, id: bitstring;
      clcheck(clsign(m, x, partial(s, id)), m, id, pubx(x), pubm(s)) = true.   (* id = (identity, X) *)
```

B.2 C1 against an outsider (Type I)

```
(* C1 against a Dolev-Yao outsider (Type-I: may choose its own keys, does not know s).
   Phase 1 leaks every long-term secret (x of the UAV, x and d of the TUAV, even s) to test
   forward secrecy of session data sent in phase 0.
   Run:  proverif -lib common c1_typeI.pv *)

free s: master [private].
free xU: secretv [private].
free xA: secretv [private].
free idU: bitstring.
free idA: bitstring.
free gk: bitstring [private].            (* group key GK_a *)
free payload: bitstring [private].       (* data a member later sends under GK_a *)

event uavRequest(bitstring, G, G).
event tuavAccept(bitstring, G, G, G).
event tuavSentKey(bitstring, G, G).
event uavGotKey(bitstring, G, G).

(* the TUAV accepts the honest UAV's REGISTERED key only if that UAV signed (Girault level 2) *)
query ei: G, ea: G; inj-event(tuavAccept(idU, pubx(xU), ei, ea)) ==> inj-event(uavRequest(idU, ei, ea)).
(* same pseudonym, ANY key: expected false under a malicious TA (key replacement, inherent to CL) *)
query X: G, ei: G, ea: G; event(tuavAccept(idU, X, ei, ea)) ==> event(uavRequest(idU, ei, ea)).
query ei: G, ea: G; inj-event(uavGotKey(idU, ei, ea)) ==> inj-event(tuavSentKey(idU, ei, ea)).
query attacker(gk) phase 0.
```

```
query attacker(payload).                    (* includes phase 1: forward secrecy *)

let UAV =
  in(c, (ea: G, ts: bitstring, sa: bitstring));
  if clcheck(sa, (idA, ea, ts), (idA, pubx(xA)), pubx(xA), pubm(s)) = true then
  new ei: exponent;
  let Ei = exp(g, ei) in
  new ts2: bitstring;
  event uavRequest(idU, Ei, ea);
  out(c, (idU, pubx(xU), Ei, ts2, clsign((idA, ea, Ei, ts2), xU, partial(s, (idU, pubx(xU))))));
  in(c, ct: bitstring);
  let K = kdf(exp(ea, ei), (idU, idA, ts2)) in
  let gkr = sdec(ct, K) in
  event uavGotKey(idU, Ei, ea);
  out(c, senc(payload, kdf2(K, gkr))).

let TUAV =
  new ea: exponent;
  let Ea = exp(g, ea) in
  new ts: bitstring;
  out(c, (Ea, ts, clsign((idA, Ea, ts), xA, partial(s, (idA, pubx(xA))))));
  in(c, (id: bitstring, X: G, Ei: G, ts2: bitstring, sig: bitstring));
  if clcheck(sig, (idA, Ea, Ei, ts2), (id, X), X, pubm(s)) = true then
  event tuavAccept(id, X, Ei, Ea);
  let K = kdf(exp(Ei, ea), (id, idA, ts2)) in
  event tuavSentKey(id, Ei, Ea);
  out(c, senc(gk, K)).

process
  out(c, pubm(s)); out(c, pubx(xU)); out(c, pubx(xA));
  ( (!UAV) | (!TUAV)
  | phase 1; out(c, (xU, xA, s)) )
```

B.3 C1 against a malicious TA (Type II)

```
(* C1 against a malicious / breached TA (Type-II): the attacker knows the master key s,
   hence every partial key d = partial(s, id), but not the UAV's secret value xU.
   Expected: authentication of the honest UAV still holds (escrow-free), and data protected by
   the pairwise key K_i stays secret. Group-key secrecy is NOT expected here: a TA can always
   enrol a phantom member (limitation L5), and ProVerif should find that attack.
   Run:  proverif -lib common c1_typeII.pv *)

free s: master.                             (* public: the TA is the adversary *)
free xU: secretv [private].
free xA: secretv [private].
free idU: bitstring.
free idA: bitstring.
free gk: bitstring [private].
free unicast: bitstring [private].          (* sent by the UAV under its pairwise key K *)

event uavRequest(bitstring, G, G).
event tuavAccept(bitstring, G, G, G).

(* the TUAV accepts the honest UAV's REGISTERED key only if that UAV signed (Girault level 2) *)
query ei: G, ea: G; inj-event(tuavAccept(idU, pubx(xU), ei, ea)) ==> inj-event(uavRequest(idU, ei, ea)).
(* same pseudonym, ANY key: expected false under a malicious TA (key replacement, inherent to CL) *)
query X: G, ei: G, ea: G; event(tuavAccept(idU, X, ei, ea)) ==> event(uavRequest(idU, ei, ea)).
query attacker(unicast).
query attacker(gk).                         (* expected to be FALSE: phantom enrolment *)

let UAV =
  in(c, (ea: G, ts: bitstring, sa: bitstring));
  if clcheck(sa, (idA, ea, ts), (idA, pubx(xA)), pubx(xA), pubm(s)) = true then
  new ei: exponent;
  let Ei = exp(g, ei) in
  new ts2: bitstring;
  event uavRequest(idU, Ei, ea);
  out(c, (idU, pubx(xU), Ei, ts2, clsign((idA, ea, Ei, ts2), xU, partial(s, (idU, pubx(xU))))));
  in(c, ct: bitstring);
  let K = kdf(exp(ea, ei), (idU, idA, ts2)) in
  let gkr = sdec(ct, K) in
  out(c, senc(unicast, kdf2(K, idU))).
```

```
let TUAV =
  new ea: exponent;
  let Ea = exp(g, ea) in
  new ts: bitstring;
  out(c, (Ea, ts, clsign((idA, Ea, ts), xA, partial(s, (idA, pubx(xA))))));
  in(c, (id: bitstring, X: G, Ei: G, ts2: bitstring, sig: bitstring));
  if clcheck(sig, (idA, Ea, Ei, ts2), (id, X), X, pubm(s)) = true then
  event tuavAccept(id, X, Ei, Ea);
  let K = kdf(exp(Ei, ea), (id, idA, ts2)) in
  out(c, senc(gk, K)).

process
  out(c, pubx(xU)); out(c, pubx(xA));
  ( (!UAV) | (!TUAV) )
```

B.4 Rekeying

```
(* RK1: rekeying after a member is removed.
   Member 1 is revoked and fully compromised (its pairwise key K1 is given to the attacker).
   Expected: the old key gk1 is known (it was a member), the new key gk2 is not.
   Run:  proverif -lib common c1_revocation.pv *)

free K1: key [private].
free K2: key [private].
free K3: key [private].
free gk1: bitstring [private].
free gk2: bitstring [private].
free v1: bitstring.
free v2: bitstring.

query attacker(gk1).                    (* expected TRUE (sanity check of the model) *)
query attacker(gk2).                    (* expected FALSE: revocation works *)

process
  (* version 1: three members *)
  out(c, (senc((gk1, v1), K1), senc((gk1, v1), K2), senc((gk1, v1), K3)));
  (* member 1 is revoked and compromised *)
  out(c, K1);
  (* RK1: fresh key for the members that stay *)
  out(c, (senc((gk2, v2), K2), senc((gk2, v2), K3)))
```

B.5 C3 handover

```
(* C3: ticket-based handover between TUAV a and TUAV b.
   The member M shares K_M with TUAV a; TUAV a and b share K_ab (from HOO).
   TUAV b keeps a table of used ticket nonces (single-use tickets).
   Phase 1 leaks K_ab and K_M to test forward secrecy of the new session.
   Run:  proverif -lib common c3_handover.pv *)

free KM: key [private].
free Kab: key [private].
free idM: bitstring.
free idA: bitstring.
free idB: bitstring.
free gkB: bitstring [private].
free payload: bitstring [private].

table usedN(bitstring).

event memberStart(bitstring, G).
event tuavBAccept(bitstring, G).
event tuavBSent(bitstring, G, G).
event memberDone(bitstring, G, G).

query e: G; inj-event(tuavBAccept(idM, e)) ==> inj-event(memberStart(idM, e)).
query e: G, eb: G; inj-event(memberDone(idM, e, eb)) ==> inj-event(tuavBSent(idM, e, eb)).
query attacker(gkB) phase 0.
```

```
query attacker(payload).

(* HO1: TUAV a issues a ticket over the existing channel K_M *)
let TUAVa =
  new tsh: bitstring;
  new N: bitstring;
  let hk = kdf2(KM, (idB, tsh)) in
  let tk = senc((idM, hk, N), Kab) in
  out(c, senc((tsh, tk), KM)).

(* HO2 / HO4 *)
let Member =
  in(c, blob: bitstring);
  let (tsh: bitstring, tk: bitstring) = sdec(blob, KM) in
  let hk = kdf2(KM, (idB, tsh)) in
  new e: exponent;
  let E = exp(g, e) in
  event memberStart(idM, E);
  out(c, (idA, tk, E, mac((tk, E), hk)));
  in(c, (Eb: G, ct: bitstring));
  let K2 = kdf2(hk, k2b(kdf(exp(Eb, e), idM))) in
  let gk = sdec(ct, K2) in
  event memberDone(idM, E, Eb);
  out(c, senc(payload, K2)).

(* HO3 *)
let TUAVb =
  in(c, (=idA, tk: bitstring, E: G, t: bitstring));
  let (id: bitstring, hk: key, N: bitstring) = sdec(tk, Kab) in
  if t = mac((tk, E), hk) then
  get usedN(=N) in 0 else
  insert usedN(N);
  event tuavBAccept(id, E);
  new eb: exponent;
  let Eb = exp(g, eb) in
  let K2 = kdf2(hk, k2b(kdf(exp(E, eb), id))) in
  event tuavBSent(id, E, Eb);
  out(c, (Eb, senc(gkB, K2))).

process
  ( (!TUAVa) | (!Member) | (!TUAVb)
  | phase 1; out(c, (KM, Kab)) )
```

B.6 The base scheme's credential

```
(* The base scheme's credential (Step US2), abstracted symbolically.
   Gamma_i = gam(s_i, ID_i, xi_i, upsilon_i, R_tu, Q_tu): every argument except s_i is public
   (xi_i = h1(r_i) is sent in clear), and the TA stores s_i.
   Scenario: the TA is honest-but-curious or breached, i.e. the attacker learns s_i.
   Expected: ProVerif finds an attack on authentication (Finding 2: key escrow, no non-repudiation).
   Run:  proverif base_escrow.pv *)

free c: channel.
free si: bitstring [private].
free idU: bitstring.

fun h(bitstring): bitstring.
fun gam(bitstring, bitstring, bitstring, bitstring): bitstring.

event uavSent(bitstring, bitstring).
event tuavAccepts(bitstring, bitstring).

query xi: bitstring; event(tuavAccepts(idU, xi)) ==> event(uavSent(idU, xi)).

let UAV =
  in(c, (rtu: bitstring, qtu: bitstring));
  new r: bitstring;
  let xi = h(r) in
  event uavSent(idU, xi);
  out(c, (idU, xi, gam(si, xi, rtu, qtu))).

(* GA1-GA4: the TUAV + TA accept iff Gamma matches what s_i determines *)
```

```
let Verifier =
  new rtu: bitstring; new qtu: bitstring;
  out(c, (rtu, qtu));
  in(c, (=idU, xi: bitstring, G: bitstring));
  if G = gam(si, xi, rtu, qtu) then
  event tuavAccepts(idU, xi).

process
  (!UAV) | (!Verifier) | out(c, si)      (* the TA's database leaks s_i *)
```

B.7 Raw results

```
== c1_typeI.pv
RESULT inj-event(tuavAccept(idU[],pubx(xU[]),ei_1,ea_2)) ==> inj-event(uavRequest(idU[],ei_1,ea_2)) is true.
RESULT event(tuavAccept(idU[],X_1,ei_1,ea_2)) ==> event(uavRequest(idU[],ei_1,ea_2)) is true.
RESULT inj-event(uavGotKey(idU[],ei_1,ea_2)) ==> inj-event(tuavSentKey(idU[],ei_1,ea_2)) is true.
RESULT not attacker(gk[]) is true.
RESULT not attacker_p1(payload[]) is true.
== c1_typeII.pv
RESULT inj-event(tuavAccept(idU[],pubx(xU[]),ei_1,ea_2)) ==> inj-event(uavRequest(idU[],ei_1,ea_2)) is true.
RESULT event(tuavAccept(idU[],X_1,ei_1,ea_2)) ==> event(uavRequest(idU[],ei_1,ea_2)) is false.
RESULT not attacker(unicast[]) is true.
RESULT not attacker(gk[]) is false.
== c1_revocation.pv
RESULT not attacker(gk1[]) is false.
RESULT not attacker(gk2[]) is true.
== c3_handover.pv
RESULT inj-event(tuavBAccept(idM[],e_1)) ==> inj-event(memberStart(idM[],e_1)) is false.
RESULT inj-event(memberDone(idM[],e_1,eb_1)) ==> inj-event(tuavBSent(idM[],e_1,eb_1)) is true.
RESULT not attacker(gkB[]) is true.
RESULT not attacker_p1(payload[]) is true.
== base_escrow.pv
RESULT event(tuavAccepts(idU[],xi_2)) ==> event(uavSent(idU[],xi_2)) is false.
```